Strategy synthesis of LIA-definable impartial combinatorial games

Liangda Fang[a,d], Jiahao Luo[a], Meihong Yang[a], Sijie Yin[a], Shiguang Feng[b], Liping Xiong[c], Quanlong Guan[a]

[a] *Jinan University, Guangzhou, 510632, Guangdong, China*
[b] *Sun Yat-sen University, Guangzhou, 510006, Guangdong, China*
[c] *Wuyi University, Jiangmen, 529020, Guangdong, China*
[d] *Pazhou Lab, Guangzhou, 510335, Guangdong, China*

Abstract

Strategy representation and reasoning has recently received much attention in artificial intelligence. Impartial combinatorial games (ICGs) are a type of elementary and fundamental games in game theory. One of the challenging problems of ICGs is to construct winning strategies, particularly, generalized winning strategies for possibly infinitely many instances of ICGs. In this paper, we investigate synthesizing generalized winning strategies for ICGs. To this end, we first propose a logical framework to formalize ICGs based on the linear integer arithmetic fragment of numeric part of PDDL. We then propose two approach to generating the candidate formula that exactly captures the states in which the player can force to win: one adapts an enumerative method, and the other improves efficiency by reducing the synthesis problem as an exact binary classification. Furthermore, we compute winning strategies for ICGs based on the winning formula. Experimental results on several games demonstrate the effectiveness of our approach.

Keywords: Impartial Combinatorial Games, Strategy Synthesis, Partial MaxSAT, Linear Integer Arithmetic

1. Introduction

Strategy representation and reasoning has recently received much attention in artificial intelligence, particularly, multi-agent systems and game theory

[1, 2, 3]. In the area of game theory, one class of the elementary and fundamental games is impartial combinatorial games (ICGs) that studies sequential games with perfect information [4]. Various ICGs (*e.g.*, Nim [5], Wythoff [6], Chomp [7] and Empty-and-Divide [8]), and their variants and extensions [9, 10] have been proposed and analyzed. The following is Take-away-3 game [11].

Example 1. *Initially, a pile of chips is on a table. In a move, a player can take at most* 3 *chips from the table and must take at least* 1 *chip. The loser is the player who has no chip can be taken.* □

In general, an ICG contains infinite number of states. Infinite-state ICGs pose two challenging problems. The first problem is to identify the exact characteristic of the set of winning states, that are the states in which the player can force to win. This problem has always been a long-standing topic of ICGs [12, 9, 13, 14]. For example, a player can force to win in any state where the number v of remaining chip is not divisible by 4 (more precisely, $v \not\equiv_4 0$) in Take-away-3 game. It is impossible that a player have a winning strategy in other states where $v \not\equiv_4 0$.

The second problem is to synthesize generalized winning strategies for infinite many winning states. An ICG instance is an ICG together with a specific initial state. A winning strategy for an ICG instance can be computed via backward search algorithm [15]. However, this strategy holds for only some instances but not (infinitely) many instances. This is illustrated in the Take-away-3 game as follows. Assume that the number of chips is initially 7. In the first round, the first player takes 3 chips. If the other player takes 1 (resp. 2 and 3) chips, then she chooses to take 3 (resp. 2 and 1) chips in the next round. Following the above strategy, the first player finally wins no matter how the other player does. But for another instance where the initial state is $v = 5$, after the first player takes 3 chips, the number of the remaining chips is 2. Then, the other player takes 2 chips and no chips is on the table. Finally, the loser is the first player. However, the first player can force to win in the initial state $v = 5$ since $5 \not\equiv_4 0$. Hence, the above strategy does not work for some instances.

In this paper, we propose an automatic approach to solving the above two problems of infinite-state ICGs. The contributions of this paper contain:

1. We propose a logical framework to formalize ICGs via the linear integer arithmetic fragment of Planning Domain Definition Language (PDDL) [16].
2. We define an arithmetic formula, called the winning formula, that is a symbolic representation that exactly captures the states in which the player forces to win. We propose a counterexample-guided approach to synthesizing the winning formula. It first attempts to generate a candidate formula ϕ via a finite set of states, and then verifies the correctness of ϕ. If ϕ passes the verification, then the algorithm terminates with the winning formula ϕ. Otherwise, a set of new states that are witnesses of its failure will be provided, and used to create a new candidate formula. The above steps repeat until the winning formula is discovered.
3. One crucial task of the counterexample-guided approach is to generate a candidate formula that is consistent with two generated sets of winning states and losing states. We design two methods to accomplish this task. The first method we adopt is a simple enumerative approach proposed in [17]. The main idea is to generate candidate formulas by induction on the size until it finds a desired one. However this approach does not scale well since exhaustive generation of all candidate formulas becomes time consuming when the size of the winning formula is relatively large. To accelerate the generation of candidate formulas, we propose the second method, and formulate it as an exact binary classification problem with all atoms as classification attributes and encode each classification problem as an instance of the partial MaxSAT problem. We can efficiently acquire the winning formula of large size with the aid of off-the-shelf partial MaxSAT solvers.
4. The another core task of the counterexample-guided approach is to check if the candidate formula is the winning formula. We design several con-

straints for the winning formula that is represented by linear integer arithmetic. The candidate formula is correct iff all of the constraints are valid. Therefore, this task can be accomplished by the state-of-the-art SMT solvers: Z3 [18], CVC4 [19] and MathSMT 5 [20].

5. To synthesize the winning strategy, we develop a counterexample-guided approach from a generated winning formula. It first produces a finite set of actions that transforms at least one winning state to a losing state, and then determines the condition formula that captures the set of winning states from which performing the action leads to a losing state for each action. A rule is a pair of an action and its corresponding condition formula, and a strategy consists of a set of rules. To check the correctness of a candidate strategy, we design the constraints for each of its rule.

6. We establish a standard benchmark of ICGs composed of 5 categories of ICGs: Subtraction, Nim, Wythoff, Welter, and Chomp for a total of $3,720$ test cases. Each test case is formalized in a PDDL file. We also evaluate our proposed approach on this benchmark, demonstrating that the effectiveness of our proposed approach.

This paper is an extended and revised version of [21] and [22]. We provide the complete proofs of all propositions, lemmas, and theorems in [22]. In [21], the strategy synthesis approach first splits it into a set of arithmetic terms such that their disjunction is equivalent to the winning formula according to a set of simple division rules. Then, for each term, it uses the enumerative algorithm to obtain the desired action such that the player reaches a losing state from any state satisfying the term via performing this action. However, it is not guaranteed that each term has a proper action, and the above approach fails to compute a winning strategy. In addition, the experimental results show that the strategy synthesis approach proposed in [22] combined with the partial MaxSAT-based method to generate candidate formulas outperforms the one in [21]. Hence, this paper does not include the illustration and experimental comparison of the strategy synthesis approach proposed in [21].

(Structure ??) The rest of the paper is organized as follows. In Section 2, we provide some necessary preliminaries, including the concepts of linear integer arithmetic (LIA), the maximum satisfiability problem, and ICGs. In Section 3, we propose a logical framework to formalize ICGs via the linear integer arithmetic fragment of Planning Domain Definition Language (PDDL) [16]. In Section 4, we introduce the exact binary classification problem and propose two approach that reduce this problem to the enumerative approach proposed in [17] and partial MaxSAT problems, respectively. In Section 5, we propose a counterexample-guided inductive synthesis (CEGIS) approach to synthesizing winning formulas and strategies for impartial combinatorial games. In Section 6, we evaluate our proposed approaches on the publicly available benchmark composed of 5 categories of ICGs: Subtraction, Nim, Wythoff, Welter, and Chomp for a total of $3,720$ test cases. (??) Finally, we offer related works in Section 7, and close with conclusions and future works in Section 8.

2. Preliminaries

In this section, we introduce the syntax of linear integer arithmetic (LIA), the partial maximum satisfiability (MaxSAT) problem, and impartial combinatorial games (ICGs).

2.1. Linear integer arithmetic

Let $\mathcal{N}$ be the set of integers and $\mathcal{V}$ the set of variables. *Linear integer arithmetic expressions* (`Exp`), *atoms* (`Atom`) and *formulas* (`Form`) are defined by the following grammar:

$$e, e_1, e_2 \in \texttt{Exp} :: c \mid v \mid e_1 + e_2 \mid e_1 - e_2$$

$$l \in \texttt{Atom} :: e_1 = e_2 \mid e_1 < e_2 \mid e_1 \leq e_2 \mid e \equiv_{c_1} c_2$$

$$\phi, \phi_1, \phi_2 \in \texttt{Form} :: \top \mid \bot \mid l \mid \neg\phi \mid \phi_1 \wedge \phi_2 \mid \phi_1 \vee \phi_2 \mid \forall v\phi \mid \exists v\phi$$

where $c, c_1, c_2 \in \mathcal{N}$ and $v \in \mathcal{V}$.

The atom $e \equiv_{c_1} c_2$ means that e and c_2 are congruence modulo c_1, that is, $e - c_2$ is divisible by c_1. We use $e_1 \neq e_2$ for $\neg(e_1 = e_2)$, $e_1 > e_2$ for $\neg(e_1 \leq e_2)$, $e_1 \geq e_2$ for $\neg(e_1 < e_2)$, $e \not\equiv_{c_1} c_2$ for $\neg(e \equiv_{c_1} c_2)$, and $\phi_1 \rightarrow \phi_2$ for $\neg\phi_1 \vee \phi_2$. We use $|e|$ (resp. $|\phi|$) to represent the size of an LIA expression e (resp. LIA formula ϕ), that is, the number of occurrences of integers, variables and connectives in e (resp. ϕ). For example, the formula $x = 3$ has size 3. Given a set Ψ of atoms, we say a formula ϕ is based on Ψ, iff it is built from the set Ψ, the connectives $\neg, \vee, \wedge, \rightarrow$ and two Boolean constants $\top$ and $\bot$. For example, suppose that $\Psi = \{x = 1, x > 2\}$. The formula $\neg(x = 1 \vee x > 2)$ is based on Ψ, but $x \leq 3 \wedge x > 2$ is not as the atom $x \leq 3 \notin \Psi$.

It is well-known that LIA allows for quantifier elimination, that is, any LIA formula can be transformed into an equivalent quantifier-free formula [23, 24]. Let $\mathcal{U}$ be a subset $\{u_1, \cdots, u_n\}$ of variables. We use $\texttt{Exp}_{\mathcal{U}}$ for the set of expressions over $\mathcal{U}$. We use $\texttt{Form}_{\mathcal{U}}$ for the set of formulas of which free variables are $\mathcal{U}$. We use $\forall\mathcal{U}\phi$ for $\forall u_1 \cdots \forall u_n \phi$ and $\exists\mathcal{U}\phi$ for $\exists u_1 \cdots \exists u_n \phi$.

We say a set S of n-tuples of integers is *LIA-definable*, iff there is an LIA formula $\phi(x_1, \cdots, x_n)$ s.t. for every n-tuple $(z_1, \cdots, z_n)$ of integers, we have $(z_1, \cdots, z_n) \in S$ iff $\phi(z_1, \cdots, z_n)$ is valid. Presburger [25] established a fundamental connection between LIA-definable sets and periodicity of the set in one-dimensional case[1]. A set S of integers is *ultimately periodic*, iff (1) there are two integers $n_l < 0$ and $p_l > 0$ s.t. for every $x \leq n_l$, $x \in S \cap \{z \mid z \leq n_l\}$ iff $x - p_l \in S \cap \{z \mid z \leq n_l\}$, and (2) there are two integers $n_u > 0$ and $p_u > 0$ s.t. for every $x \geq n_u$, $x \in S \cap \{z \mid z \geq n_u\}$ iff $x + p_u \in S \cap \{z \mid z \geq n_u\}$.

Theorem 1 ([25]). *A set S of integers is LIA-definable iff S is ultimately periodic.*

For example, the set of even integers has period 2 and is therefore LIA-definable, which can be represented by $x \equiv_2 0$. However, the set of prime

[1]The original theorems in [25, 26] is stated over the natural numbers. However, it can be extended to the integers.

numbers is not LIA-definable.

A set S of n-tuples of integers is *linear*, iff there are a finite set $\vec{z}_1, \cdots, \vec{z}_n$ of n-tuples of integers s.t. $\vec{z} \in S$ iff $\vec{z} + \lambda_1 \vec{z}_1 + \cdots + \lambda_n \vec{z}_n \in S$ for all integers λ_i where $1 \leq i \leq n$. A set S of n-tuples of integers is *semi-linear*, iff it is a finite union of linear sets. Ginsburg and Spanier [26] showed that semi-linear sets exactly captures LIA-definable sets.

Theorem 2 ([26])**.** *A set S of n-tuples of integers is LIA-definable iff S is semi-linear.*

2.2. Partial maximum satisfiability

Given a propositional formula in conjunctive normal form, the *satisfiability (SAT)* problem aims to determine if there is an assignment that satisfies all of the clauses in the formula. The *maximum satisfiability (MaxSAT)* problem, the optimization variation of the SAT problem, searches a truth assignment that satisfies the maximum number of clauses. One of the significant generalizations of the MaxSAT problem is the *partial MaxSAT* problem. It divides the clauses into two types: *hard* and *soft*, and searches an assignment that satisfies all of the hard clauses and maximizes the number of satisfied soft clauses.

2.3. Impartial combinatorial games

In this section, we briefly introduce *impartial combinatorial games (ICGs)*. A game that satisfies the following conditions is called an ICG [4]: (1) There are two players and many states such that the player can move from one state to another one. (2) Two players alternate moving and have the same choice of moving. (3) The game ends when it moves to an *ending state* in which no player has a possible move. An *instance* of an ICG is the ICG together with an initial state. In general, an ICG may contain infinite states, and hence it may have infinite instances. An ICG is *terminating*, iff each of its instances always ends in a finite number of moves. The problem of deciding if an ICG is terminating is undecidable [27]. Unless stated otherwise, we assume that ICGs are terminating in the remaining of this paper.

Each instance of an ICG is a finite two-player game with perfect information and no-chance move where the players take turns moving. In addition, any terminating ICG does not end in a draw. By Zermelo's Theorem [28], there always exists a winning strategy for one player in an ICG.

It can be easily observed that the player cannot guarantee to win in all states. To distinguish states in which the player can forces to win from those in which she cannot, we classify states into two types: *winning* and *losing states*.

Definition 1 ([4])**.** In an ICG, the sets $\mathcal{W}$ of winning states and $\mathcal{L}$ of losing states are recursively defined as

1. All ending states are losing states.
2. All states such that there is at least one move to a losing state are winning states.
3. All states such that the only possible moves are to winning states are losing states.

Definition 1 provides the exact characteristics of winning and losing states in a recursive way. In the base case, Condition 1 requires all legal ending states to be losing states. Both Conditions 2 and 3 are induction steps. Condition 2 says that every state such that at least one applicable move results in a losing state is a winning state. Condition 3 means that every state such that all possible successor states are winning states is a losing state. The sets $\mathcal{W}$ of winning states and $\mathcal{L}$ of losing states are disjoint.

The above definition characterizes the concepts of losing states and winning states under the normal play rule. Under the misère rule, the definitions of winning and losing states are the same as Definition 1 except that all ending states are winning states. For the sake of clarity, throughout this paper, we assume that ICGs follow the normal play rule unless explicitly stated otherwise. Our approach to constructing winning strategies can be easily lifted to the case under the misère rule and will be discussed in Section 6.

We hereafter introduce the concept of *strategy*. ICGs are a special case of Gale-Stewart games, and so are determined by memoryless strategies [29]. In

addition, it is impossible to obtain a winning strategy for the player in a losing state, there is no need to consider how to choose a move in this case. We hence define a *strategy* as a function from winning states to moves. A strategy is *winning*, iff every winning state leads to a losing state following the move defined in this strategy.

3. A logical framework

In this section, we first introduce the concepts of numeric states and actions, and then present a logical framework for describing ICGs based on numeric states and actions. The framework we adopt is a slight modification of the linear integer arithmetic fragment of the PDDL 2.1 language defined in [16].

3.1. Numeric states and actions

Throughout this paper, we use the notion $\mathcal{X}$ for the set of state variables that are used to define numeric states. An expression e (resp. a formula ϕ) is *semi-ground*, if $e \in \mathtt{Exp}_{\mathcal{X}}$ (resp. $\phi \in \mathtt{Form}_{\mathcal{X}}$). For example, suppose that $\mathcal{X} = \{v_1, v_2\}$. The expression $v_1 - 1$ is semi-ground, but $v_2 + k$ is not since k is not a state variable.

A *numeric state* s is an interpretation that maps every state variable v to an integer $v(s)$. For a state s, we can evaluate a semi-ground expression e into an integer $e(s)$ that the expression simplifies to when replacing each state variable v with its corresponding value $v(s)$. The Boolean value $\phi(s)$ of a semi-ground formula ϕ can be defined in a similar way. A numeric state s *satisfies* a formula ϕ, denoted by $s \models \phi$, iff $\phi(s) = \top$. A formula ϕ *implies* another one ψ, denoted by $\phi \models \psi$, iff $s \models \psi$ for every state s satisfying ϕ. A formula ϕ is *valid*, iff $s \models \phi$ for every state s; otherwise, it is *invalid*.

Each *numeric action* $a(\mathcal{K})$ is parameterized by a vector $\mathcal{K}$ of numeric variables, and is defined by a pair $\langle \mathtt{pre}, \mathtt{eff} \rangle$ where $\mathtt{pre} \in \mathtt{Form}_{\mathcal{X} \cup \mathcal{K}}$ denotes the precondition and `eff` is the effect represented by a set of numeric effects. A *numeric*

effect is a triple $\langle \phi, v, e \rangle$ where $\phi \in \texttt{Form}_{\mathcal{X}\cup\mathcal{K}}$, $v \in \mathcal{X}$, and $e \in \texttt{Exp}_{\mathcal{X}\cup\mathcal{K}}$. Intuitively, it means that if ϕ holds in the state s, then the value of v becomes $e(s)$ after executing the action; otherwise, it remains unchanged. Given an action $a(\mathcal{K})$, we use $\texttt{pre}(a(\mathcal{K}))$ and $\texttt{eff}(a(\mathcal{K}))$ for its precondition and effect, respectively.

Let $a(\mathcal{K})$ be an action and Σ a vector of semi-ground expressions where $\mathcal{K}$ and Σ have the same length n. The notation $e[\Sigma]$ (resp. $\phi[\Sigma]$) is the semi-ground expression (resp. formula) obtained from $e \in \exp_{\mathcal{X}\cup\mathcal{K}}$ (resp. $\phi \in \texttt{Form}_{\mathcal{X}\cup\mathcal{K}}$) by replacing the i-th parameter k_i of $\mathcal{K}$ with the i-th semi-ground expression e_i of Σ for $1 \leq i \leq n$. A *semi-ground action* $a[\Sigma]$ is defined as $\langle \texttt{pre}, \texttt{eff} \rangle$ where $\texttt{pre}(a[\Sigma]) = \texttt{pre}(a(\mathcal{K}))[\Sigma]$ and $\langle \phi[\Sigma], v, e[\Sigma] \rangle \in \texttt{eff}(a[\Sigma])$ for every $\langle \phi, v, e \rangle \in \texttt{eff}(a(\mathcal{K}))$. A semi-ground action $a[\Sigma]$ is *applicable* in a state s, iff $s \models \texttt{pre}(a[\Sigma])$. The result of applying an applicable action $a[\Sigma]$ in a state s is a successor state, written $\tau(s, a[\Sigma])$, where every variable v is associated to an integer $v(\tau(s, a[\Sigma]))$ defined as:

$$v(\tau(s, a[\Sigma])) = \begin{cases} e[\Sigma](s), & \text{if there is } \langle \phi[\Sigma], v, e[\Sigma] \rangle \in \texttt{eff}(a[\Sigma]) \text{ and } \phi[\Sigma](s) = \top; \\ v(s), & \text{otherwise.} \end{cases}$$

We require that every action $a(\mathcal{K})$ is not *self-contradictory*, formally, for every state s and every vector Σ of semi-ground expressions, if $\langle \phi_1[\Sigma], v_1, e_1[\Sigma] \rangle$, $\langle \phi_2[\Sigma], v_2, e_2[\Sigma] \rangle \in \texttt{eff}(a[\Sigma])$ s.t. $\phi_1[\Sigma](s) = \phi_2[\Sigma](s) = \top$, then $e_1[\Sigma](s) = e_2[\Sigma](s)$.

For brevity, we sometimes use a for (semi-ground) actions.

3.2. Formalization

Definition 2. An LIA-definable game is defined as a tuple $\Pi = \langle \mathcal{X}, \mathcal{A}, \mathcal{C}, \mathcal{E} \rangle$ where

- $\mathcal{X}$: a finite set of state variables.
- $\mathcal{A}$: a finite set of numeric actions.
- $\mathcal{C}$: a formula of $\texttt{Form}_{\mathcal{X}}$ denoting all legal states.
- $\mathcal{E}$: a formula of $\texttt{Form}_{\mathcal{X}}$ denoting all ending states.

To obtain the finite representation of winning states, we use an arithmetic formula to exactly capture this notion, called *the winning formula*.

Definition 3. Let $\Pi = \langle \mathcal{X}, \mathcal{A}, \mathcal{C}, \mathcal{E} \rangle$ be an ICG. We say a formula $\phi \in \mathtt{Form}_{\mathcal{X}}$ is a *winning formula* of Π, iff for every state s, $s \models \mathcal{C} \wedge \phi$ iff s is a legal winning state.

The winning formula ϕ along with the formula $\mathcal{C}$ exactly characterizes the set of winning states. In general, there are many logical different winning formulas for an ICG. But they are logically equivalent under the background theory $\mathcal{C}$, that is, $\mathcal{C} \models \phi \equiv \phi'$. Hence, we sometimes call a winning formula the winning one. Since the sets of winning states and losing states are disjoint, $\neg\phi$ is the symbolic representation of losing states.

It has been shown that some ICGs formalized in LIA have LIA-definable set of winning states. For example, the set of winning states of every Subtraction game and its variant i-Mark game is proved to be ultimately periodic [30, 13]. It follows from [25] that every Subtraction as well as i-Mark game has LIA-definable set of winning states. However, not all ICGs formalized in LIA admit LIA-definable winning formulas. Krusemeyer [31] proposed Mark game with the following formalization:

- $\mathcal{X} : \{v\}$;
- $\mathcal{A} : \{take\}$ where
  - $\mathtt{pre}(take(k)) : v > 0 \wedge v - k \geq 0 \wedge (k = 1 \vee 2k = v \vee 2k = v + 1)$;
  - $\mathtt{eff}(take(k)) : \{\top, v, v - k\}$;
- $\mathcal{C} : v \geq 0$;
- $\mathcal{E} : v = 0$.

In Mark game, the player can take either 1 or $\lceil \frac{v}{2} \rceil$ chips from the table, where v denotes the number of remaining chips. As proved in [32], the set of winning states in Mark game is not ultimately periodic. Hence, its winning formula is

not LIA-definable. In addition, the problem of deciding if a LIA-definable ICG admits LIA-definable winning formula is undecidiable (ref ??).

In this paper, we focus on the synthesis of LIA-definable winning formulas and leave the synthesis of winning formulas that require more expressive languages to future work.

A *rule* (ψ, a) is a pair of semi-ground formulas ψ and semi-ground actions a. Given a rule (ψ, a), we call ψ the *condition formula* of (ψ, a). A *strategy* δ is a set of rules.

Definition 4. Let $\Pi = \langle \mathcal{X}, \mathcal{A}, \mathcal{C}, \mathcal{E} \rangle$ be an ICG and ϕ a winning formula of Π.

- A rule (ψ, a) is *winning*, iff for every state s satisfying $\mathcal{C} \wedge \phi \wedge \psi$, we have a is applicable in s and $\tau(s, a)$ is a losing state.
- A rule (ψ, a) is *complete*, iff for every state s s.t. $\tau(s, a)$ is a losing state, we have $\psi(s) = \top$.
- A rule (ψ_i, a_i) is *redundant* w.r.t. a strategy $\{(\psi_1, a_1), \cdots, (\psi_n, a_n)\}$, iff every state s satisfying ψ_i, there is a formula ψ_j s.t. $i \neq j$ and $\psi_j(s) = \top$.
- A strategy $\{(\psi_1, a_1), \cdots, (\psi_n, a_n)\}$ is *winning*, iff every rule (ψ_i, a_i) is winning.
- A strategy $\{(\psi_1, a_1), \cdots, (\psi_n, a_n)\}$ is *exhaustive*, iff for every winning state s of Π, there is a formula ψ_i s.t. $\psi_i(s) = \top$.

The winning property of the rule (ψ, a) requires that executing the action a in every state satisfying $\mathcal{C} \wedge \phi \wedge \psi$ yields a losing state. The completeness property of the rule says that the formula ψ covers all of the states s s.t. its successor state $\tau(s, a)$ is a losing state. The redundancy property of the rule (ψ_i, a_i) w.r.t. the strategy means that all condition formulas of the other rules of δ covers the condition formula of ψ_i. The winning property of the strategy means that each of its rule is a winning rule. The exhaustiveness property of the strategy stipulates that each winning state has its corresponding winning forced action.

Example 2. *The formalization of the Take-away-3 game [11] is given as follows. The state variable v denotes the number of the remaining chips in this game. For simplicity, we use an integer c to denote the numeric state where v is assigned to c. A move consists of taking any number (from 1 to m) of chips. Here we consider the case where $m = 3$. The following are the definition of ICG and the description of actions.*

- $\mathcal{X} : \{v\}$;
- $\mathcal{A} : \{take(k)\}$;
- $\mathcal{C} : v \geq 0$;
- $\mathcal{E} : v = 0$;
- $\texttt{pre}(take(k)) : v \geq k \wedge k > 0 \wedge k \leq 3$;
- $\texttt{eff}(take(k)) : \{\langle \top, v, v - k \rangle\}$.

The two numeric states 0 and 4 are legal losing states. The former is a legal ending state. The latter has three applicable semi-ground actions: $take(1)$, $take(2)$, and $take(3)$. Executing each of them yields three different winning states $\tau(4, take(1)) = 3$, $\tau(4, take(2)) = 2$, and $\tau(4, take(3)) = 1$.

The winning formula of this game is $v \not\equiv_4 0$. It means that any legal state in which the number of the remaining chips is not a multiple of 4 is a winning state. The three rules $(v \equiv_4 1, take(1)), (v \equiv_4 2, take(2))$ and $(v \equiv_4 3, take(3))$ are winning and complete rules, and form a winning and exhaustive strategy. But $(v = 2, take(2))$ is a winning but incomplete rule as there is a state 6 from which taking 2 chips leads to a losing state, which does not satisfy $v = 2$. □

4. Synthesizing winning formulas

In this section, we propose a counterexample-guided inductive synthesis (CEGIS) approach to synthesizing winning formulas for ICGs over infinite states, illustrated in Algorithm 1. A counterexample in this approach in fact is a state in ICGs.

Algorithm 1: `SynWinForm`(Π)

Input: $\Pi = \langle \mathcal{X}, \mathcal{A}, \mathcal{C}, \mathcal{E} \rangle$: an ICG.

Output: ϕ: the winning formula of Π

```
1 Initialize the set S of legal states
2 P, N ← ClassifyStates(S)
3 while true do
4     φ ← GenerateCandidate(P, N)
5     S' ← VerifyWinForm(φ)
6     if S' = ∅ then return φ ;
7     else
8         P', N' ← ClassifyStates(S')
9         P ← P ∪ P' and N ← N ∪ N'
```

4.1. The main algorithm

The main idea of Algorithm 1 is as follows: It first attempts to generate a candidate formula ϕ via a set of counterexamples, and then verifies the correctness of ϕ. If ϕ passes the verification, then the algorithm terminates with the winning formula ϕ. Otherwise, a set of new counterexamples will be gathered and used to create a new candidate formula. The above steps repeat until the algorithm discovers the winning formula of the ICG.

We hereafter elaborate Algorithm 1 in details. It initializes the set S as a finite set of legal states, which is accomplished by an SMT solver (Line 1). Each legal state s is the state satisfying the formula $\mathcal{C}$ denoting all legal states (cf. Definition 2). The SMT solver takes the formula $\mathcal{C}$ as input, and returns a state satisfying $\mathcal{C}$. However, the SMT solver often returns the same state for the formula $\mathcal{C}$. We can easily fix this defect and generate many different legal states via adding an extra subformula on $\mathcal{C}$. Suppose that we are given a set S' of legal states. The formula $\mathcal{C}_{S'}$ is $\mathcal{C} \wedge \bigwedge_{s \in S'} \neg[\bigwedge_{v \in \mathcal{X}} (v = v(s))]$, and hence any state satisfying $\mathcal{C}_{S'}$ is a legal state not in S'.

The set S of states will be classified into two sets $\mathcal{P}$ of positive states and $\mathcal{N}$ of negative states (Line 2). In an ICG, each positive state is a winning state

$\mathcal{P}$	$\mathcal{N}$	Candidate formulas	Counterexamples
1	0	$v = 1$	2
$1, 2$	0	$v > 0$	4
$1, 2$	$0, 4$	$v > 0 \wedge v < 3$	3
$1, 2, 3$	$0, 4$	$v > 0 \wedge v \leq 3$	5
$1, 2, 3, 5$	$0, 4$	$v \not\equiv_4 0$	$\emptyset$

Table 1: A run of synthesizing the winning formula

while each negative state is a losing state.

Algorithm 1 then enters a loop until a winning formula is discovered (Lines 3 - 9). With the two sets of positive states and of negative states in hand, we can produce a candidate winning formula ϕ (Line 4). Then, we will check whether the candidate formula ϕ is a winning formula (Line 5). If ϕ is correct (that is, $S' = \emptyset$), then Algorithm 1 terminates with the solution ϕ (Line 6). Otherwise, the verification procedure yields some states of S' that are witnesses of its failure and incorporates them into $\mathcal{P}$ and $\mathcal{N}$, respectively (Lines 8 - 9).

Example 3. *Table 1 shows the run of synthesizing the winning formula of Take-away-3 game. At the beginning, only state* 1 *is in the set* $\mathcal{P}$ *and state* 0 *in* $\mathcal{N}$*. Our approach generates the formula* $v = 1$ *as the candidate solution that is consistent with* $\mathcal{P}$ *and* $\mathcal{N}$*. However, it is not the correct winning formula that holds for all the legal states and the SMT solver returns a winning state* 2*. Our approach takes into account the new state. Three candidate formulas,* $v > 0$*,* $v > 0 \wedge v < 3$*, and* $v > 0 \wedge v \leq 3$*, are found sequentially. However, all of the aforementioned three formulas fail verification. The corresponding counterexamples are the losing state* 4 *and the winning states* 3 *and* 5*. Finally, the algorithm terminates with the correct winning formula* $v \not\equiv_4 0$*, which is constructed from the above* 6 *states.* □

Finally, three subprocedures of Algorithm 1 remain to be addressed: (1) classifying winning states from losing states (Line 2), (2) generating a candi-

date formula (Line 4), and (3) verifying the correctness of a candidate formula (Line 5). These will be resolved in the subsequent three sections.

4.2. Classification of numeric states

In this subsection, we design an algorithm to the problem of classifying states into winning and losing states that works in a recursive way based on Definition 1.

We first provide the notion of *transition formulas* and *global transition formulas*. The definition of transition formulas is a numerical generalization of the existing propositional encoding for the planning problem which focuses on transition systems over finite states [33, 34]. The transition formula $\mathcal{T}(a(\mathcal{K}))$ for an action $a(\mathcal{K})$ reflects the mapping from predecessor states to successor states due to the effect of $a(\mathcal{K})$. To distinguish the predecessor state variable and the successor one, we use the unprimed variable v for the former and the primed one v' for the latter.

Definition 5. The transition formula $\mathcal{T}(a(\mathcal{K}))$ of $a(\mathcal{K})$ is the conjunction of the following formulas:

1. $\texttt{pre}(a(\mathcal{K}))$;
2. $\bigwedge_{\langle\phi,v,e\rangle\in\texttt{eff}(a(\mathcal{K}))}(\phi \rightarrow v' = e)$;
3. $\bigwedge_{v\in\mathcal{V}}[(\bigvee_{\langle\phi,v,e\rangle\in\texttt{eff}(a(\mathcal{K}))}\phi) \vee v' = v]$.

The first formula is the precondition of action $a(\mathcal{K})$. The second one is the effect axiom meaning that the value of v becomes $e(s)$ if the condition ϕ holds in the state s. The third formula is the frame axiom meaning that if no condition ϕ s.t. $\langle\phi, v, e\rangle \in \texttt{eff}(a)$ holds, then the value of v remains unchanged.

The global transition formula $\mathcal{T}(\mathcal{A})$ is the union of the mapping due to the effect of every action $a(\mathcal{K}) \in \mathcal{A}$, denoting the state transition system of the whole game. The definition is as follows:

Definition 6. Let $\mathcal{A}$ be a set of actions. The global transition formula $\mathcal{T}(\mathcal{A})$ of Π is $\bigvee_{a(\mathcal{K})\in\mathcal{A}} \exists\mathcal{K}[\mathcal{T}(a(\mathcal{K}))]$.

Algorithm 2: `IsWinningState`(Π, s)

Input: $\Pi = \langle \mathcal{X}, \mathcal{A}, \mathcal{C}, \mathcal{E} \rangle$: an LIA-definable ICG;
s: a legal state.

Output: $\top$, if s is a winning state; $\bot$, otherwise.

```
1  if Cache(s) ≠ nil then return Cache(s) ;
2  if s ⊨ C ∧ E then
3      Cache(s) ← ⊥
4      return ⊥
5  foreach legal successor state s' of s do
6      if IsWinningState(Π, s') = ⊥ then
7          Cache(s) ← ⊤
8          return ⊤
9  Cache(s) ← ⊥
10 return ⊥
```

The following example illustrates how transition formulas are constructed from the definition of actions.

Example 4. *Let us continue the Take-away-3 game. We construct the transition formula $\mathcal{T}(take(k))$ as follows. The precondition is $v \geq k \wedge k > 0 \wedge k \leq 3$. The effects contain one elements: $\langle \top, v, v - k \rangle$. Hence, the effect axiom of $\mathcal{T}(take(k))$ is $(\top \rightarrow v' = v - k)$. We now consider the frame axiom of $\mathcal{T}(take(k))$. For variable v, the formula is $\top \vee v' = v$. Conjoining the above formulas and simplifying the conjunction lead to the transition formula:*

$$\mathcal{T}(take(k)) \equiv v \geq k \wedge k > 0 \wedge k \leq 3 \wedge v' = v - k.$$

By Definition 6, the global transition formula

$$\mathcal{T}(\mathcal{A}) \equiv \exists k(v \geq k \wedge k > 0 \wedge k \leq 3 \wedge v' = v - k).$$

□

Algorithm 2 provides a recursive procedure for determining whether a legal state s in an LIA-definable ICG is a winning state. The algorithm employs

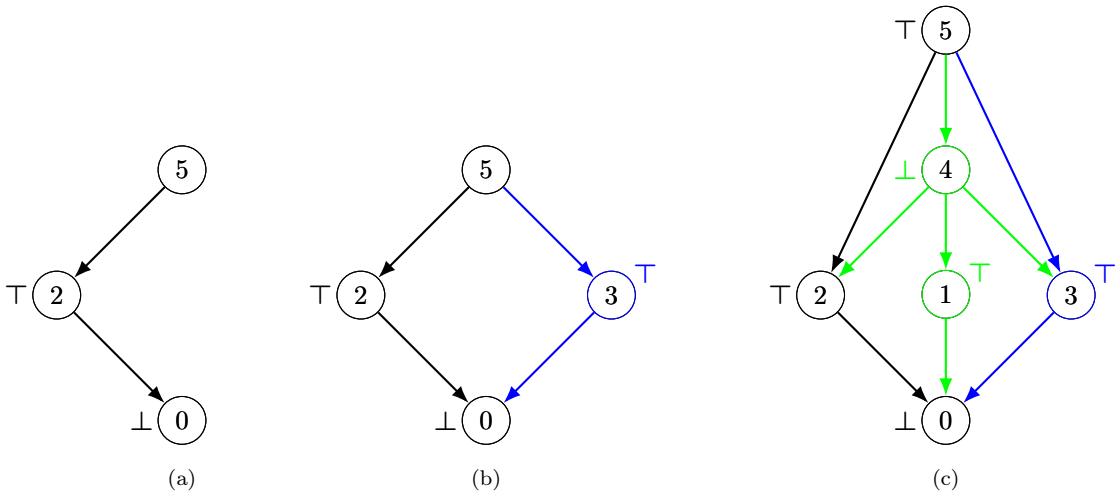


Figure 1: The run of Algorithm 2 in Take-away-3 game with initial state 5.

memorization to avoid redundant computation by first checking if the result for state s has already been cached (Line 1). It then verifies if s is a legal ending state (Lines 2 - 4). If it is the case, then s is classified as a losing states since all ending states are losing states (cf. Definition 1). Otherwise, Algorithm 2 iterates over every legal successor state s' of s (Lines 5 - 8). If at least one successor state s' is a losing state, then s is a winning state since all states such that there is at least one move to a losing state are winning states. Conversely, s is a losing state since all states such that the only possible moves are to winning states are losing states (Lines 9 - 10).

The following example illustrates the run of Algorithm 2.

Example 5. *Let us consider the Take-away-3 game at initial state 5. Figure 1 illustrates a run of Algorithm 2 on this game. In this figure, an arrow from state s to state s' denotes that s' is a legal successor state of s. The label beside a state records the value stored in the cache: $\top$ means that it is winning, while $\bot$ means that it is losing.*

Since the value of state 5 has not yet been computed and it is not an ending state, Algorithm 2 iterates through its successor states 2, 3 and 4. State 2, also

not previously computed and not an ending state, leads to one of its successor state 0. *As state* 0 *is an ending state, it is immediately classified as losing and stored in the cache. It follows that state* 2 *is a winning state, and it is unnecessary to traverse the remaining successor state* 1 *(Figure 1(a)).*

Then, Algorithm 2 evaluates state 3 *by examining its successor states* 0, 1 *and* 2. *Since state* 0 *was already known as a losing state in memorization, state* 3 *is winning (Figure 1(b)). Similarly, state* 1 *is identified as winning.*

For state 4, *because all of its successor states* 1, 2 *and* 3 *are winning, state* 4 *itself is classified as losing. Finally, state* 5 *has the losing successor state* 4, *it is classified as winning, as shown in Figure 1(c).* □

4.3. Generation of candidate formulas

In this subsection, we propose two approaches for generating candidate formulas. The first approach adopts a simple enumerative algorithm proposed in [17]. The second approach formulate the task as exact binary classification problem and encode each classification problem as an instance of the partial MaxSAT problem.

4.3.1. Enumeration-based approach

The key to the enumeration algorithm is generating candidate formulas (or expressions) by induction on size according to the grammar illustrated in Section 2.1. In the first step, only atomic arithmetic expressions of size 1 (*i.e.*, state variables and integers) are considered. As the set of integers is infinite, the integers considered in the first step are 0, 1 together with the constants explicitly occurring in the formalization of games. Firstly, this restriction does not limit the enumeration of all integers since any positive integer c can be expressed as the sum of c 1's, and the negative integer $-c$ equals to $0-c$. Furthermore, we observe that integers appearing in the winning formula and strategy are often close to some explicit constants defined in the formalization. Hence, including these domain-specific constants can accelerate the synthesis process. In the k-th step,

we generate arithmetic expressions and formulas of size k. The enumeration process terminates when it finds candidates formula ϕ such that $\{s \mid s \models \phi \wedge \mathcal{C}\} = \mathcal{P}$.

To improve the efficiency of the enumeration process, we remove expressions and formulas based on the notion of indistinguishability. Formally, two formulas ϕ_1 and ϕ_2 are *indistinguishable* w.r.t. a set S of states, if $\phi_1(s) = \phi_2(s)$ for every $s \in S$. The definition of indistinguishability between two expressions is similar. For example, suppose that $S = \{(v_1 : 1, v_2 : 2), (v_1 : 2, v_2 : 2)\}$. It is easily verified that the two formulas $v_1 \neq v_2$ and $v_1 + 1 = v_2$ hold in the first state and do not hold in the second one. Therefore, the two formulas are indistinguishable w.r.t. S. Suppose that we are generating expressions and formulas of size k. If ϕ_1 and ϕ_2 are indistinguishable w.r.t. S, then it is sufficient to carry forward only one of ϕ_1 and ϕ_2 to the next iteration $k + 1$. Based on the notion of indistinguishability, we can prune the search space of candidate formulas.

4.3.2. Exact binary classification-based approach

We observe that enumeration-based approach does not scale well since exhaustive generation of all candidate formulas becomes time consuming when the size of the winning formula increases. To accelerate the generation process, we introduce the exact binary classification problem and propose an approach that encodes this problem as a partial MaxSAT problem.

In contrast to enumeration-based approach, we first generate a set Ψ of arithmetic atoms rather than formulas of increasing sizes until for every pair of positive state and negative state is separated by at least one atom $\psi \in \Psi$. In order to prune the huge size of Ψ, we also eliminate some arithmetic atoms in Ψ based on the notion of indistinguishability. Based on the set Ψ of arithmetic atoms, we reduce the problem of searching for candidate formulas to the exact binary classification problem.

Definition 7. Given two sets $\mathcal{P}$ of positive states and $\mathcal{N}$ of negative states and a set Ψ of arithmetic atoms, the task of the exact binary classification problem is to generate a formula ϕ based on Ψ s.t. $\phi(s) = \top$ for every state $s \in \mathcal{P}$ and $\phi(s) = \bot$ for every state $s \in \mathcal{N}$.

The exact binary classification problem may not have a solution in some cases. The sufficient and necessary condition for the existence of a solution is established by the following theorem.

Theorem 3. *Let $\mathcal{P}$ and $\mathcal{N}$ be two sets of positive and negative states, respectively, and Ψ a set of atoms. The following two statements are equivalent:*

1. *For each pair of positive state s and negative state t, there is an atom $\psi \in \Psi$ s.t. $\psi(s) \neq \psi(t)$.*
2. *There exists a formula ϕ based on Ψ that classifies $\mathcal{P}$ from $\mathcal{N}$.*

To prove the above theorem, we need the following two definitions that are used to synthesize a desired formula for the exact binary classification problem if the solution exists.

Definition 8. Let $\mathcal{P}$ and $\mathcal{N}$ be two sets of positive and negative states, respectively, and Ψ a set of atoms. The *filter set* $\Psi_{\mathcal{P},\mathcal{N}}$ of Ψ w.r.t. $\mathcal{P}$ and $\mathcal{N}$ is $\{\psi \mid \psi \in \Psi \text{ and } \psi(s) \neq \psi(t) \text{ for some } s \in \mathcal{P} \text{ and } t \in \mathcal{N}\}$.

The filter set is the subset of Ψ in which each element classifies at least one positive state and one negative state.

Definition 9. Let S be a set of states and Ψ a set of atoms. The *characteristic formula* of S based on Ψ is $\bigvee_{s \in S} \left[\left(\bigwedge_{\psi \in \Psi \text{ and } \psi(s) = \top} \psi \right) \wedge \left(\bigwedge_{\psi \in \Psi \text{ and } \psi(s) = \bot} \neg\psi \right) \right]$.

The conjunct $\left(\bigwedge_{\psi \in \Psi \text{ and } \psi(s) = \top} \psi \right) \wedge \left(\bigwedge_{\psi \in \Psi \text{ and } \psi(s) = \bot} \neg\psi \right)$ can be considered as a truth assignment of the state s on the set Ψ of atoms. The characteristic formula collects the assignments of every state $s \in S$ on Ψ.

Proof of Theorem 3: ($\Leftarrow$): Let ϕ be a formula based on Ψ s.t. ϕ classifies $\mathcal{P}$ from $\mathcal{N}$. We prove the first statement by contradiction. Let $s \in \mathcal{P}$ and $t \in \mathcal{N}$ s.t. no atom $\psi \in \Psi$ satisfies the property $\psi(s) \neq \psi(t)$. It follows that $\phi(s) = \phi(t)$. This contradicts the assumption.

($\Rightarrow$): Let ϕ be the characteristic formula of $\mathcal{P}$ based on $\Psi_{\mathcal{P},\mathcal{N}}$. Clearly, ϕ is also based on Ψ. By Definition 9, for every positive state s, ϕ contains $\left(\bigwedge_{\psi \in \Psi \text{ and } \psi(s) = \top} \psi \right) \wedge \left(\bigwedge_{\psi \in \Psi \text{ and } \psi(s) = \bot} \neg\psi \right)$. Hence, $s \models \phi$. By the first

statement of Theorem 3 and Definition 8, for every negative state t, there is an atom $\psi \in \Psi_{\mathcal{P},\mathcal{N}}$ and a positive state s s.t. $\psi(t) \neq \psi(s)$. None of the conjuncts of ϕ is satisfied by t. Hence, $t \not\models \phi$. In summary, ϕ classifies $\mathcal{P}$ from $\mathcal{N}$. □

The proof of Theorem 3 provides a direct construction for a formula that discriminates positive states from negative ones. However, the given set Ψ in general includes a large number of arithmetic atoms. So is the filter set $\Psi_{\mathcal{P},\mathcal{N}}$. Therefore, the characteristic formula of $\mathcal{P} \cup \mathcal{N}$ based on $\Psi_{\mathcal{P},\mathcal{N}}$ is too complex to serve as a practical candidate formula.

To decrease the amount of atoms in the solution, we devise a practical method to the exact binary classification problem based on partial MaxSAT. The propositional encoding $T(\mathcal{P}, \mathcal{N}, \Psi)$ takes as inputs two sets $\mathcal{P}$ of positive states and $\mathcal{N}$ of negative states and the set Ψ of arithmetic atoms. The propositions in $T(\mathcal{P}, \mathcal{N}, \Psi)$ are

- $Sel(\psi)$: the atom $\psi \in \Psi$ appears in the formula ϕ.

The hard constraints of $T(\mathcal{P}, \mathcal{N}, \Psi)$ are

- $\bigvee_{\psi(s) \neq \psi(t)} Sel(\psi)$ for every $s \in \mathcal{P}$ and $t \in \mathcal{N}$.

It means that for every pair of positive state s and negative state t, at least one atom of Ψ that distinguishes between s and t is chosen, corresponding to the first statement of Theorem 3.

The clauses in the soft constraints of $T(\mathcal{P}, \mathcal{N}, \Psi)$ are the negative literal $\neg Sel(\psi)$ for each atom ψ. The partial MaxSAT solver looks for a solution with the fewest possible arithmetic atoms.

Example 6. *We use Take-away-3 game to illustrate the partial MaxSAT-based method to learn the characteristic formula. Suppose that we are given the sets of positive states* $\mathcal{P} : \{1, 2, 3, 5\}$ *and of negative states* $\mathcal{N} : \{0, 4\}$*, and the set of arithmetic atoms* $\Psi : \{v = 0, v = 1, v < 1, v < 4, v \leq 0, v \equiv_2 0, v \equiv_3 1, v \equiv_4 0\}$*. The Boolean values of the above atomic formulas is shown in Table 2. The propositional encoding* $T(\mathcal{P}, \mathcal{N}, \Psi)$ *includes* 8 *propositions, one for each atomic*

States		$v=0$	$v=1$	$v<1$	$v<4$	$v\leq 0$	$v\equiv_2 0$	$v\equiv_3 1$	$v\equiv_4 0$
$\mathcal{P}$	1	$\bot$	$\top$	$\bot$	$\top$	$\bot$	$\bot$	$\top$	$\bot$
	2	$\bot$	$\bot$	$\bot$	$\top$	$\bot$	$\top$	$\bot$	$\bot$
	3	$\bot$	$\bot$	$\bot$	$\top$	$\bot$	$\bot$	$\bot$	$\bot$
	5	$\bot$	$\bot$	$\bot$	$\bot$	$\bot$	$\bot$	$\bot$	$\bot$
$\mathcal{N}$	0	$\top$	$\bot$	$\top$	$\top$	$\top$	$\top$	$\bot$	$\top$
	4	$\bot$	$\bot$	$\bot$	$\bot$	$\bot$	$\top$	$\top$	$\top$

Table 2: The example of learning the characteristic formula

formula in Ψ*. For the positive state* 1 *and the negative state* 0*, the hard constraint is* $Sel(v = 0) \vee Sel(v = 1) \vee Sel(v < 1) \vee Sel(v \leq 0) \vee Sel(v \equiv_2 0) \vee Sel(v \equiv_3 1) \vee Sel(v \equiv_4 0)$ *since only the above* 7 *atoms distinguish state* 0 *from* 1*. The hard constraints for other pairs of positive state and negative state can be similarly constructed. The soft constraints are the negations of these* 8 *propositions of* $T(\mathcal{P}, \mathcal{N}, \Psi)$*. The partial MaxSAT solver returns an assignment in which only* $Sel(v \equiv_4 0)$ *is* $\top$ *and the other* 7 *propositions are* $\bot$*.*

From the above assignment, only the atom $v \equiv_4 0$ *is picked from the assignment. As the chosen atom does not hold in every state of* $\mathcal{P}$*, its negation* $v \not\equiv_4 0$ *is the characteristic formula of* $\mathcal{P}$ *based on* Ψ*. In addition, it exactly classifies the given states of positive states and of negative states.* □

4.4. Correctness verification

In this subsection, we present the constraints for the winning formula that is used to verify the correctness of candidate formula based on the notion of global transition formulas (cf. Definition 6).

Definition 10. Let $\Pi = \langle \mathcal{X}, \mathcal{A}, \mathcal{C}, \mathcal{E} \rangle$ be an ICG. The constraints for the winning formula ϕ of Π are as follows:

1. $\mathcal{C} \wedge \mathcal{E} \rightarrow \neg\phi$;
2. $(\mathcal{C} \wedge \phi) \rightarrow \exists \mathcal{X}'[\mathcal{T}(\mathcal{A}) \wedge (\mathcal{C} \wedge \neg\phi)[\mathcal{X}/\mathcal{X}']]$;
3. $(\mathcal{C} \wedge \neg\phi) \rightarrow \forall \mathcal{X}'[\mathcal{T}(\mathcal{A}) \rightarrow (\mathcal{C} \wedge \phi)[\mathcal{X}/\mathcal{X}']]$.

where $\psi[\mathcal{X}/\mathcal{X}']$ is the formula obtained by replacing every occurrence of $v \in \mathcal{X}$ in ψ with v'.

Each of the above constraints corresponds to each condition of winning and losing states (cf. Definition 1). As a result, we obtain the soundness theorem of the constraints for the winning formula.

Theorem 4. *Let $\Pi = \langle \mathcal{X}, \mathcal{A}, \mathcal{C}, \mathcal{E} \rangle$ be an ICG. A formula ϕ is the winning formula of Π iff all of the constraints illustrated in Definition 10 are valid.*

Proof. ($\Rightarrow$): Assume that ϕ is the winning formula. It follows that every state s satisfying $\mathcal{C} \wedge \phi$ is winning and every state s satisfying $\mathcal{C} \wedge \neg\phi$ is losing. We prove the three constraints illustrated in Definition 10 are valid.

1. $\mathcal{C} \wedge \mathcal{E} \rightarrow \neg\phi$: Let s be a state satisfying $\mathcal{C} \wedge \mathcal{E}$. It follows that s is a legal ending state. By the first item of the definition of winning and losing states (cf. Definition 1), s is a losing state. Hence, $s \models \neg\phi$ and the first constraint is valid.
2. $(\mathcal{C} \wedge \phi) \rightarrow \exists \mathcal{X}'[\mathcal{T}(\mathcal{A}) \wedge (\mathcal{C} \wedge \neg\phi)[\mathcal{X}/\mathcal{X}']]$: Let s be a state satisfying $\mathcal{C} \wedge \phi$. It follows that s is a legal winning state. By the second item of the definition of winning and losing states, there is at least one applicable action a in s s.t. $\tau(s, a)$ is a losing state . Hence, $s \models \exists \mathcal{X}'[\mathcal{T}(\mathcal{A}) \wedge (\mathcal{C} \wedge \neg\phi)[\mathcal{X}/\mathcal{X}']]$ and the second constraint is valid.
3. $(\mathcal{C} \wedge \neg\phi) \rightarrow \forall \mathcal{X}'[\mathcal{T}(\mathcal{A}) \rightarrow (\mathcal{C} \wedge \phi)[\mathcal{X}/\mathcal{X}']]$: The proof of the validity of the third constraint is the same as that of the second constraint by using the third item of the definition of winning and losing states.

($\Leftarrow$): Assume that ϕ is a formula s.t. all of the constraints illustrated in Definition 10 is valid. We define a relation $\prec$ between states in the following:

$$s' \prec s \text{ iff } s' \text{ is a successors state of } s.$$

Since Π is terminating, the game always ends in a finite number of moves from any legal state. Hence, $\prec$ contains no infinite descending chains and is well founded [35]. Let S be the legal states of Π. It follows that $(S, \prec)$ is a well

founded set. We prove that every state s satisfying $\mathcal{C} \wedge \phi$ is winning and every state s satisfying $\mathcal{C} \wedge \neg\phi$ is losing by induction on $(S, \prec)$.

Base case (s is a legal ending state): It follows that $s \models \mathcal{C} \wedge \mathcal{E}$. By the first constraint $\mathcal{C} \wedge \mathcal{E} \rightarrow \neg\phi$, we get that $s \models \neg\phi$. By the definition of winning formula (cf. Definition 3), s is a losing state.

Inductive step (s is a legal but not ending state): Suppose that $s \models \phi$. By the second constraint $(\mathcal{C} \wedge \phi) \rightarrow \exists \mathcal{X}'[\mathcal{T}(\mathcal{A}) \wedge (\mathcal{C} \wedge \neg\phi)[\mathcal{X}/\mathcal{X}']]$, there is a successor state $\tau(s, a)$ of s s.t. $\tau(s, a) \models \mathcal{C} \wedge \neg\phi$. By the inductive assumption, $\tau(s, a)$ is a losing state. By the second item of the definition of winning and losing states, s is a winning state. Similarly, we can prove that s is a losing state when $s \models \neg\phi$ by using the third constraint $(\mathcal{C} \wedge \neg\phi) \rightarrow \forall \mathcal{X}'[\mathcal{T}(\mathcal{A}) \rightarrow (\mathcal{C} \wedge \phi)[\mathcal{X}/\mathcal{X}']]$ and the third item of the definition of winning and losing states. □

Theorem 4 reduces the correctness verification problem of an LIA-formula for an LIA-definable ICG to the validity problem of an LIA-formula. Thanks to the decidability of the validity problem of LIA [25], the above correctness verification problem is decidable.

Corollary 1. *The problem of deciding if an LIA-formula is a winning formula for an LIA-definable ICG is decidable.*

In addition, Theorem 4 confirms the decidability of correctness verification of an LIA-formula and the soundness property of Algorithm 1 for synthesizing the winning formula.

Corollary 2. *Let* $\Pi = \langle \mathcal{X}, \mathcal{A}, \mathcal{C}, \mathcal{E} \rangle$ *be an ICG. The formula returned by Algorithm 1 is the winning formula of* Π.

Proof sketch: Lines 5 of Algorithm 1 checks if candidate formula ϕ is the winning formula according to the constraints illustrated in Definition 10. If $S' = \emptyset$, then the constraints for the winning formula are valid. By Theorem 4, ϕ is the winning formula. □

Algorithm 3: `SynWinStrat`$(\Pi, \phi, \mathcal{P}, \mathcal{N})$

Input: $\Pi = \langle \mathcal{X}, \mathcal{A}, \mathcal{C}, \mathcal{E} \rangle$: an LIA-definable ICG;

ϕ: the winning formula of the Π;

$\mathcal{P}$: the set of positive states;

$\mathcal{N}$: the set of negative states.

Output: δ: a winning and exhaustive strategy

$\delta^c, \delta^w, \mathcal{A}^f, \mathcal{A}^u \leftarrow \emptyset$

$\mathcal{P}' \leftarrow \mathcal{P}$

while true do

  $\mathcal{P}'' \leftarrow \mathcal{P}'$ and $\mathcal{P}' \leftarrow \emptyset$

  $\mathcal{A}' \leftarrow$ `EnumerateActions`$(\mathcal{P}'', \mathcal{A}^f \cup \mathcal{A}^u)$

  foreach $a \in \mathcal{A}^u \cup \mathcal{A}'$ **do**

    if $a \notin \mathcal{A}^u$ **then**

      $\mathcal{P}_a \leftarrow \{s \in \mathcal{P} \mid s \models \texttt{pre}(a)$ and $\tau(s, a) \in \mathcal{L}\}$

      $\mathcal{N}_a \leftarrow \mathcal{N} \cup \{s \in \mathcal{P} \mid s \not\models \texttt{pre}(a)$ or $\tau(s, a) \notin \mathcal{L}\}$

    else `/* `$a \in \mathcal{A}^u$` */`

      $\mathcal{P}_a \leftarrow \mathcal{P}_a \cup \{s \in \mathcal{P}'' \mid s \models \texttt{pre}(a)$ and $\tau(s, a) \in \mathcal{L}\}$

      $\mathcal{N}_a \leftarrow \mathcal{N}_a \cup \{s \in \mathcal{P}'' \mid s \not\models \texttt{pre}(a)$ or $\tau(s, a) \notin \mathcal{L}\}$

    $\psi_a \leftarrow$ `GenerateCandidate`$(\mathcal{P}_a, \mathcal{N}_a)$

    $S^w_a \leftarrow$ `VerifyWinningRule`(ψ_a, a)

    if $S^w_a \neq \emptyset$ **then**

      $\mathcal{A}^u \leftarrow \mathcal{A}^u \cup \{a\}$ and $\mathcal{P}' \leftarrow \mathcal{P}' \cup S^w_a$

    else

      $S^c_a \leftarrow$ `VerifyCompleteRule`(ψ_a, a)

      if $S^c_a = \emptyset$ **then**

        $\mathcal{A}^f \leftarrow \mathcal{A}^f \cup \{a\}$ and $\mathcal{A}^u \leftarrow \mathcal{A}^u \setminus \{a\}$

        $\delta^c \leftarrow \delta^c \cup \{(\psi_a, a)\}$ and $\delta^w \leftarrow \delta^w \setminus \{(\psi'_a, a)\}$ `/* `(ψ'_a, a)` is the existing incomplete but winning rule of `δ^w` */`

      else

        $\mathcal{A}^u \leftarrow \mathcal{A}^u \cup \{a\}$ and $\mathcal{P}' \leftarrow \mathcal{P}' \cup S^c_a$

        $\delta^w \leftarrow (\delta^w \setminus \{(\psi'_a, a)\}) \cup \{(\psi_a, a)\}$

  $S^e \leftarrow$ `VerifyExhaustiveStrategy`$(\delta^c \cup \delta^w)$

  if $S^e = \emptyset$ **then return** `Simplify`$(\delta^c \cup \delta^w)$;

  else $\mathcal{P}' \leftarrow (\mathcal{P}' \cup S^e) \setminus \mathcal{P}$ and $\mathcal{P} \leftarrow \mathcal{P} \cup \mathcal{P}'$;

5. Synthesizing winning strategies

In this section, we also devise a CEGIS approach to synthesizing winning and exhaustive strategies. We first introduce the main algorithm in Algorithm 3, and then present the correctness verification of rules and strategies.

5.1. The main algorithm

The main synthesis algorithm first generate a pool of candidate actions for a set of positive states. For each action, we synthesize its condition formula via the candidate formula generation approach. Each action and its corresponding condition formula constitute a rule and a set of rules forms a strategy. Then, we check against the winning and completeness property of each rule and the winning and exhaustiveness property of the strategy. This step will continue until we synthesize a winning and exhaustive strategy. The following gives a comprehensive explanation of Algorithm 3.

After the process of synthesizing winning formulas, we obtain the winning formula ϕ along with two sets $\mathcal{P}$ of winning states and $\mathcal{N}$ of losing states, which serve as inputs to Algorithm 3. Algorithm 3 maintains the following data entities. δ^c is a set of winning and complete rules, while δ^w is a set of winning but incomplete rules. $\mathcal{A}^f$ is a set of actions a whose corresponding rule (ψ_a, a) has been verified to be both winning and complete. $\mathcal{A}^u$ is a set of actions whose corresponding rule are neither winning nor complete. $\mathcal{P}$ is the set of all generated winning states. It is initialized as the set of winning states produced by synthesizing the winning formula and is incrementally expanded during strategy synthesis. $\mathcal{P}'$ and $\mathcal{P}''$ are the sets of the newly discovered winning states at the current and previous iterations, respectively.

At the beginning, Algorithm 3 initializes δ^c, δ^w, $\mathcal{A}^f$, $\mathcal{A}^u$ and $\mathcal{P}'$ (Lines 1 - 2). It then enters a loop (Lines 3 - 27). At each iteration, it first generates a pool $\mathcal{A}'$ of new actions not in $\mathcal{A}^f \cup \mathcal{A}^u$ for $\mathcal{P}''$, which will be elaborated in Algorithm 4 (Line 5). For each state $s \in \mathcal{P}''$, the algorithm first checks whether some action in $\mathcal{A}^f \cup \mathcal{A}^u$ is applicable in s and leads to a losing state. If so, then no additional action is enumerated for s; otherwise, the algorithm search a new action for s.

For every action $a \in \mathcal{A}^u \cup \mathcal{A}'$, Algorithm 3 tries to generate a desired condition formula ψ_a for a, which was introduced in Section 4.3 (Lines 6 - 24). We collect the set $\mathcal{P}_a$ of positive states and $\mathcal{N}_a$ of negative states for action a (Lines 7 - 12). The set $\mathcal{P}_a$ is a subset of $\mathcal{P}$ s.t. each state $s \in \mathcal{P}$ satisfies the following conditions: (1) the action a is applicable in s; and (2) the successor state $\tau(s, a)$ is losing. In contrast, the set $\mathcal{N}_a$ includes all states in $\mathcal{N}$ together with those states in $\mathcal{P}$ that do not satisfy the above two conditions. If action a is a new action, then both $\mathcal{P}_a$ and $\mathcal{N}_a$ are constructed from $\mathcal{P}$ and $\mathcal{N}$ (Lines 7 - 9). Otherwise, $\mathcal{P}_a$ and $\mathcal{N}_a$ have been constructed before and are enlarged only with the states in $\mathcal{P}''$ (Lines 10 - 12).

The algorithm then generates a candidate formula ψ_a that separates $\mathcal{P}_a$ from $\mathcal{N}_a$ via the two methods proposed in Section 4.3 (Line 13). The correctness of the rule (ψ_a, a) will be validated (Lines 14 - 24). We first verify the winning property of (ψ_a, a) (Line 14). If (ψ_a, a) is not winning, we are given a set S_a^w of states certifying that ψ_a is not a desired formula for the action a. Each state $s \in S_a^w$ has the following two properties: (1) s is a winning state; and (2) either a is not applicable in s, or $\tau(s, a)$ is not a losing state. Then, we add a into $\mathcal{A}^u$ and enlarge $\mathcal{P}'$ by S_a^w (Line 16). Otherwise, (ψ_a, a) is winning and we will examine its completeness property. This is because, in some cases, action a is the unique action that changes some winning states to a losing state, but ψ_a may not involve these winning states. If (ψ_a, a) is complete, then we add a into $\mathcal{A}^f$ and remove it from $\mathcal{A}^u$ (Line 20). At the same time, we add the rule (ψ_a, a) into δ^c and remove the existing incomplete but winning rule (ψ'_a, a) from δ^w (Line 21). Otherwise, we obtain a set S_a^c of states certifying that ψ_a is not a desired formula for the action a.

Then, we handle this case similarly to the case where (ψ_a, a) is not winning (Line 23). In addition, we update the condition formula of the existing rule (ψ'_a, a) in δ^w to ψ_a (Line 24).

Finally, Algorithm 3 aggregate all rules of δ^c and δ^w into a strategy δ. If δ is exhaustive, then we terminate with a simplified strategy of δ^c and δ^w (Line 26). The simplification process will repeatedly remove redundant rules of the strategy until no further rule can be removed. Otherwise, Algorithm 3 moves

Algorithm 4: `EnumerateActions`$(\Pi, \mathcal{P}, \mathcal{A}^e)$

Input: $\Pi = \langle \mathcal{X}, \mathcal{A}, \mathcal{C}, \mathcal{E} \rangle$: an LIA-definable ICG;
$\mathcal{P}$: the set of positive states;
$\mathcal{A}^e$: the set of existing semi-ground actions;

Output: $\mathcal{A}'$: the pool of semi-ground actions.

```
1  Ω ← FindFirstTerm(P) and A' ← ∅
2  foreach s ∈ P do
3      if ∃ a[Σ] ∈ A^e ∪ A' s.t. s ⊨ pre(a) and τ(s, a[Σ]) ∈ L then
4          continue;
5      Found ← ⊥
6      repeat
7          foreach a ∈ A do
8              foreach Σ ∈ Ω^k s.t. a[Σ] ∉ A^e ∪ A' do  /* k is the length of
                the parameter vector of a */
9                  if s ⊨ pre(a[Σ]) and τ(s, a[Σ]) ∈ L then
10                     A' ← A' ∪ {a[Σ]}
11                     Found ← ⊤
12                     break;
13             if Found then break;
14         Ω ← Ω ∪ FindNextTerm(P)
15     until Found;
16 return A'
```

to the next iteration with two updated sets $\mathcal{P}'$ and $\mathcal{P}$ (Line 27).

Algorithm 4 takes the set $\mathcal{P}$ of winning states together with the set $\mathcal{A}^e$ of existing semi-ground actions as input, and outputs a pool $\mathcal{A}'$ of new semi-ground actions s.t. every state $s \in \mathcal{P}$ has its associated action $a[\Sigma] \in \mathcal{A}^e \cup \mathcal{A}'$ that is applicable in s and that leads s to a losing state. Algorithm 4 maintains a set Ω of semi-ground terms so as to enumerate semi-ground actions. For each state $s \in \mathcal{P}$, it first checks if $\mathcal{A}^e \cup \mathcal{A}'$ has the associated action $a[\Sigma]$ for s (Line 3). If so, the algorithm does not enumerate any action for s (Line 4). Otherwise, it

Iter.	$\mathcal{P}$	$\mathcal{A}'$	a	$\mathcal{P}_a$	$\mathcal{N}_a$	ψ_a	S_a^w	S_a^c	$\mathcal{A}^f$	$\mathcal{A}^u$	S^e
1	$1,2,3,5$	$take(1)$ $take(2)$ $take(3)$	$take(1)$	$1,5$	$0,2,3,4$	$v \equiv_4 1$	$\emptyset$	$\emptyset$	$take(1)$	$take(2)$ $take(3)$	6
			$take(2)$	2	$0,1,3,4,5$	$v = 2$	$\emptyset$	6			
			$take(3)$	3	$0,1,2,4,5$	$v = 3$	$\emptyset$	7			
2	$6,7$	$\emptyset$	$take(2)$	$2,6$	$0,1,3,4,5,7$	$v \equiv_4 2$	$\emptyset$	$\emptyset$	$take(1)$ $take(2)$ $take(3)$	$\emptyset$	$\emptyset$
			$take(3)$	$3,7$	$0,1,2,4,5,6$	$v \equiv_4 3$	$\emptyset$	$\emptyset$			

Table 3: A run of synthesizing the winning strategy

considers each action $a(\mathcal{K}) \in \mathcal{A}$ and searches for a vector of parameters Σ based on Ω such that $s \models \texttt{pre}(a[\Sigma])$ and $\tau(s, a[\Sigma]) \in \mathcal{L}$ (Lines 5 - 15). Once such vector is found, the corresponding action $a[\Sigma]$ is added to $\mathcal{A}'$ (Line 9 -12). If no desired semi-ground action is generated for s, then the set Ω of semi-ground terms will be enlarged (Line 14).

Example 7. *Table 3 shows the synthesis process of the winning strategy for the Take-away-3 game. According to example, we have the set of winning states* $\mathcal{P} = \{1,2,3,5\}$ *and of losing states* $\mathcal{N} = \{0,4\}$.

In the first iteration, we enumerate the set of actions $\mathcal{A}' = \{take(1), take(2), take(3)\}$. *For action* $take(1)$*, the set* $\mathcal{P}_{take(1)}$ *of positive states for the condition formula is* $\{1,5\}$ *while the set* $\mathcal{N}_{take(1)}$ *of negative states is* $\{0,2,3,4\}$*. We obtain the condition formula* $v \equiv_4 1$ *for* $take(1)$ *based on* $\mathcal{P}_{take(1)}$ *and* $\mathcal{N}_{take(1)}$*. Similarly, the condition formulas for* $take(2)$ *and* $take(3)$ *are* $v = 2$ *and* $v = 3$*, respectively. Now, we have three rules:* $(v \equiv_4 1, take(1))$*,* $(v = 2, take(2))$ *and* $(v = 3, take(3))$*. By verifying the winning and completeness properties of the above three rules, we get that the first rule is both winning and complete and hence* $take(1)$ *is added into* $\mathcal{A}^f$*. However, the latter two rules are winning but incomplete. The verification process for completeness property returns two states* 6 *and* 7 *for* $(v = 2, take(2))$ *and* $(v = 3, take(3))$*, respectively. So the two sets* $S^c_{take(2)} = \{6\}$ *and* $S^c_{take(3)} = \{7\}$*. The set* $\mathcal{A}^u$ *contains the two actions* $take(2)$ *and* $take(3)$*. After combing the above three rules into a strategy* δ*, we check if*

δ is exhaustive. The exhaustiveness check returns a state 6, meaning that δ has not an applicable action for the winning state 6 s.t. its successor state is losing.

In the second iteration, the algorithm continues with the newly generated winning states $\mathcal{P}'' = \{6, 7\}$. The existing actions take(2) and take(3) are applicable in states 6 and 7, respectively and lead them to losing state 4. So no new actions are enumerated and $\mathcal{A}' = \emptyset$. We add states 6 into $\mathcal{P}_{take(2)}$ and $\mathcal{N}_{take(3)}$ and 7 into $\mathcal{P}_{take(3)}$ and $\mathcal{N}_{take(2)}$. Based on the new sets of positive states and of negative states, we obtain the condition formulas $v \equiv_4 2$ and $v \equiv_4 3$ for take(2) and take(3), respectively. Both the two rules $(v \equiv_4 2, take(2))$ and $(v \equiv_4 3, take(3))$ are winning and complete. The above two rules together with $(v \equiv_4 1, take(1))$ form a winning and exhaustive strategy. □

5.2. Correctness verification

In this subsection, we present the constraints used to verify the winning, completeness and redundancy properties of rules, and the exhaustiveness properties of strategies.

Definition 11. Let $\Pi = \langle \mathcal{X}, \mathcal{A}, \mathcal{C}, \mathcal{E} \rangle$ be an ICG and ϕ a winning formula of Π.

- The constraint for the winning rule (ψ, a) is $(\mathcal{C} \wedge \phi \wedge \psi) \rightarrow \{\texttt{pre}(a) \wedge \forall \mathcal{X}'[\mathcal{T}(a) \rightarrow \neg\phi[\mathcal{X}/\mathcal{X}']]\}$.
- The constraint for the complete rule (ψ, a) is $\exists \mathcal{X}'[\mathcal{T}(a) \wedge \neg\phi[\mathcal{X}/\mathcal{X}']] \rightarrow \psi$.
- The constraint for the redundant rule (ψ_i, a_i) w.r.t. a strategy $\{(\psi_1, a_1), \cdots, (\psi_n, a_n)\}$ is $\psi_i \rightarrow \bigvee_{i \neq j} \psi_j$.
- The constraint for the exhaustive strategy $\{(\psi_1, a_1), \cdots, (\psi_n, a_n)\}$ is $(\mathcal{C} \wedge \phi) \rightarrow (\bigvee_{i=1}^{n} \psi_i)$.

Theorem 5. *Let $\Pi = \langle \mathcal{X}, \mathcal{A}, \mathcal{C}, \mathcal{E} \rangle$ be an ICG. Then the following holds:*

- *A rule (ψ, a) is winning iff the constraint for the winning rule illustrated in Definition 11 is valid.*

- *A rule (ψ, a) is complete iff the constraint for the complete rule illustrated in Definition 11 is valid.*
- *A rule (ψ_i, a_i) is redundant w.r.t. a strategy $\{(\psi_1, a_1), \cdots, (\psi_n, a_n)\}$ iff the constraint for the redundant rule illustrated in Definition 11 is valid.*
- *A strategy $\{(\psi_1, a_1), \cdots, (\psi_n, a_n)\}$ is exhaustive iff the constraint for the redundant rule illustrated in Definition 11 is valid.*

Proof.

- The winning property of rules: ($\Rightarrow$): Assume that (ψ, a) is a winning rule. Let s be a state satisfying $\mathcal{C} \wedge \phi \wedge \psi$. By the definition of winning rules (cf. Definition 4), a is applicable in s and $\tau(s, a)$ is a losing state. It follows that $s \models \texttt{pre}(a) \wedge \forall \mathcal{X}'[\mathcal{T}(a) \rightarrow \neg\phi[\mathcal{X}/\mathcal{X}']]$. Hence, the constraint for the winning rule is valid.

  ($\Leftarrow$): Assume that (ψ, a) is a rule s.t. the constraint for the winning rule is valid. Let s be a state satisfying $\mathcal{C} \wedge \phi \wedge \psi$. By the constraint for the winning rule, we get that $s \models \texttt{pre}(a) \wedge \forall \mathcal{X}'[\mathcal{T}(a) \rightarrow \neg\phi[\mathcal{X}/\mathcal{X}']]$. It follows that a is applicable in s and $\tau(s, a)$ is a losing state. Hence, (ψ, a) is a winning rule.

- The completeness property of rules: ($\Rightarrow$): Assume that (ψ, a) is a complete rule. Let s be a state satisfying $\exists \mathcal{X}'[\mathcal{T}(a) \wedge \neg\phi[\mathcal{X}/\mathcal{X}']]$. It follows that $\tau(s, a)$ is a losing state. By the definition of complete rules (cf. Definition 4), $s \models \psi$. Hence, the constraint for the complete rule is valid.

  ($\Leftarrow$): Assume that (ψ, a) is a rule s.t. the constraint for the complete rule is valid. Let s be a state s.t. $\tau(s, a)$ is a losing state. It follows that $s \models \exists \mathcal{X}'[\mathcal{T}(a) \wedge \neg\phi[\mathcal{X}/\mathcal{X}']]$. By the constraint for the complete rule, we get that $s \models \psi$. Hence, (ψ, a) is a complete rule.

- The redundancy property of rules: ($\Rightarrow$): Assume that (ψ_i, a_i) is a redundant rule of $\{(\psi_1, a_1), \cdots, (\psi_n, a_n)\}$. Let s be a state satisfying ψ_i. By the definition of redundant rules (cf. Definition 4), there is a formula ψ_j s.t. $i \neq j$ and $s \models \psi_i$. Hence, the constraint for the redundant rule is valid.

($\Leftarrow$): Assume that (ψ_i, a_i) is a rule s.t. the constraint for the redundant rule is valid. Let s be a state satisfying ψ_i. By the constraint for the redundant rule, $s \models \bigvee_{i \neq j} \psi_j$. It follows that there is a formula ψ_j s.t. $i \neq j$ and $s \models \psi_i$. Hence, (ψ_i, a_i) is is a redundant rule of $\{(\psi_1, a_1), \cdots, (\psi_n, a_n)\}$.

- The exhaustiveness property of strategies: ($\Rightarrow$): Assume that $\{(\psi_1, a_1), \cdots, (\psi_n, a_n)\}$ is an exhaustive strategy. Let s be a state satisfying $\mathcal{C} \wedge \phi$. By the definition of exhaustive strategies (cf. Definition 4), there is a condition formula ψ_i s.t. $s \models \psi_i$. It follows that $s \models \bigvee_{i=1}^{n} \psi_i$. Hence, the constraint for the exhaustive strategy is valid.

  ($\Leftarrow$): Assume that $\{(\psi_1, a_1), \cdots, (\psi_n, a_n)\}$ is a strategy s.t. the constraint for the exhaustive rule is valid. Let s be a legal winning state. It follows that $s \models \mathcal{C} \wedge \phi$. By the constraint for the constraint for the exhaustive strategy, $s \models \bigvee_{i=1}^{n} \psi_i$. It follows that there is a condition formula ψ_i s.t. $s \models \psi_i$. Hence, $\{(\psi_1, a_1), \cdots, (\psi_n, a_n)\}$ is exhaustive.

□

(Decidability for four properties of rules and strategies ??)

Finally, we obtain the soundness theorem of Algorithm 3.

Corollary 3. *Let $\Pi = \langle \mathcal{X}, \mathcal{A}, \mathcal{C}, \mathcal{E} \rangle$ be an ICG. The set of rules returned by Algorithm 3 is an exhaustive and winning strategy of* Π.

Proof sketch: Let δ be a set of rules $\{(\psi_1, a_1), \cdots, (\psi_n, a_n)\}$ returned by Algorithm 3.

We first prove that δ is winning. Line 14 checks whether a rule (ψ_a, a) satisfies the winning property. If $S_a^w = \emptyset$, then the constraint for the winning rule is valid. By Theorem 5, (ψ_a, a) is winning. If (ψ_a, a) is complete, then Line 21 inserts it into δ^c; otherwise, Line 24 inserts it into δ^w. All rules of $\delta^c \cup \delta^w$ are thus winning. By the definition of winning strategies (cf. Definition 4), $\delta^c \cup \delta^w$ is winning.

Next, we prove the exhaustiveness of δ. Line 25 performs the exhaustiveness check of δ. If $S^e = \emptyset$, then the constraint for the exhaustive strategy is valid.

By Theorem 5, δ is exhaustive. In addition, the simplification process only removes redundant rules (ψ', a') from $\delta^c \cup \delta^w$. By Theorem 5, $\psi' \rightarrow (\bigvee_{i=1}^{n} \psi_i)$. So $(\bigvee_{i=1}^{n} \psi_i) \equiv (\psi' \vee \bigvee_{i=1}^{n} \psi_i)$. Removing redundant rules does not affect the exhaustiveness of δ. □

6. The misère rule

In this section, we apply our approach to learning winning formulas and strategies under the misère rule. The definitions of sets of winning states and losing states are the same as Definition 1 except that all ending states are winning states. In addition, we do not allow executing an action in winning but ending states. It is easy to carry forward our approach to computing winning strategies via the following minor modifications. (1) In the step of synthesizing winning formulas, classifying legal states (Algorithm 2) into winning and losing states is based on the new definition. (2) The first and second constraints for winning formulas (cf. Definition 10) are replaced by $\mathcal{C} \wedge \mathcal{E} \rightarrow \phi$ and $(\mathcal{C} \wedge \neg\mathcal{E} \wedge \phi) \rightarrow \exists\mathcal{X}'[\mathcal{T}(\mathcal{A}) \wedge \neg\phi[\mathcal{X}/\mathcal{X}']]$, respectively. (3) The constraint for the winning rule (ψ, a) (cf. Definition 4) is replaced by $(\mathcal{C} \wedge \neg\mathcal{E} \wedge \psi) \rightarrow \{\texttt{pre}(a) \wedge \forall\mathcal{X}'[\mathcal{T}(a) \rightarrow \neg\phi[\mathcal{X}/\mathcal{X}']]\}$.

7. Experimental evaluation

This section first presents the implementation of our proposed algorithms, then elaborate the benchmarks we establish in the experiments, and finally analyzes the experimental results.

7.1. Implementation

We implemented the synthesis approach for winning formulas and strategies, illustrated in Algorithms 1 and 3 in Python 3.7. The two variants, based on different candidate formula generation methods, are referred to as SynEnum (enumeration-based) and SynMS (exact binary classification-based). Our tool takes as input a PDDL-like file that formalizes an ICG and outputs the winning

formula and strategy for this ICG. We employ Z3 [18] as the SMT solver to verify the correctness of the winning formula and strategy as well as to determine if a state is winning. We adopt NuWLS [36] as the partial MaxSAT solver in order to choose the small set of generated arithmetic atoms. NuWLS is an incomplete partial MaxSAT solver and may return a sub-optimal solution. This however has no influence on the soundness of the chosen set as the solution satisfies the hard constraint. The time-out bounds for Z3 and NuWLS are 10 and 3 seconds, respectively. The experiments were conducted on a PC with an Intel Core i7-7700 3.60 GHz CPU and 16GB RAM running under Windows 10, with a runtime cutoff of $3,600$ seconds. Source codes are provided in this link[2].

7.2. Benchmarks

We evaluate the performance of our approach SynEnum and SynMS on a large number of ICG domains spanning 5 categories: Subtraction [37], Nim [5], Wythoff [6], Welter [38] and Chomp [7]. Each category contains multiple classes of ICGs, including some extensions or variants of the original game. The PDDL file of each test case are publicly available in this link[3]. Since the winning formula for some classes of ICGs are neither LIA-definable nor known, we impose certain limitations to guarantee that these classes admit LIA-definable solution. For example, the winning formula of 3-rowed Chomp game is still unknown. We fix the maximum number of cookies on the 3rd row and obtain a restricted version of 3-rowed Chomp which has an LIA-definable solution. In summary, the benchmark consists of 32 game classes with $3,720$ test cases.

7.2.1. Subtraction games

This category includes 3 classes: Subtraction [37], Take-away [11], and S, D-Mark [13], contributing 479 test cases.

Subtraction A game consists of a pile of v chips. A player can take k chips from the pile where $k \in S$ where S is a finite set of positive integers. Each

[2]https://github.com/caro2357/SynMS
[3]https://github.com/caro2357/Domain

test case is defined by a distinct choice of the set S. We consider the set S with size at most 4 and $S \subseteq [1, 10]$. This class totally contains 385 test cases.

Take-away This class is a simplification of Subtraction games. The set S is an interval $[1, m]$. All test cases differ in the parameter m, which ranges from 1 to 30, yielding a total of 30 test cases.

S, D-Mark Given two finite sets of positive integers S and D, a player can take either k chips with $k \in S$ or $v - \frac{v}{d}$ chips when $d \in D$ divides v. All test cases have different choices of S and D where $S = [1, m]$ with $2 \leq m \leq 9$ and $D = \{d\}$ with $2 \leq d \leq 9$, yielding 64 test cases in total.

7.2.2. Nim games

This category includes the following classes: Nim [5], Monotonic Nim [10], Empty and Transfer [8], Empty All But One [8], Empty and Redistribute [8], Entropy Reduction [8], Min [39], l-Slow Nim [10], Sharing Nim [40], Small Nim [30], Greedy Nim [41], Odd-or-Even Nim [42], Modular Nim [43], Pointed Nim [41], End Nim [44], Loop End Nim [44], Staircase Nim [45], and Circular Nim [46], contributing $2,302$ test cases.

Nim There are n piles of chips where v_i denotes the number of chips of the i-th pile. Each move removes a positive number of chips from exactly one pile. This class is the canonical benchmark for positions with several piles and serves as a baseline for evaluating synthesis over multidimensional states. Because the winning formula of Nim requires exclusive-or and is therefore not LIA-definable, we restrict the number of piles to at most 3, bound the third pile by $m \leq 5$, and also include one unrestricted version, obtaining 7 test cases.

Monotonic Nim. A player may remove chips from one pile only if the resulting sequence remains non-decreasing, that is, $v_i \geq v_{i+1}$. The distinguishing feature of this class is that each move must preserve a global order relation

among heaps rather than merely reduce one component. We restrict the number of piles to at most 3, bound the third pile by $m \leq 5$, and include the unrestricted version, giving 7 test cases.

Empty and Transfer. A move empties one pile, selects another pile, and transfers some but not all of its chips into the emptied pile. Unlike standard Nim, this class emphasizes redistribution between piles and therefore captures interactions that cannot be reduced to independent decreases of individual piles. We restrict the number of piles to 3, bound the third pile by $m \leq 5$, and include the unrestricted version, obtaining 7 test cases.

Empty All But One. A move empties $n - 1$ piles and redistributes the chips of the remaining pile among all piles so that each pile remains nonempty. The key feature here is whole-position redistribution, because a legal action rewrites the entire heap configuration in one step rather than changing a single heap locally. We restrict the number of piles to 3, bound the third pile by $m \leq 5$, and obtain 6 test cases in total.

Empty and Redistribute. One pile is first discarded completely, and the chips in the remaining piles are then redistributed among all piles so that each pile contains at least one chip. This class is more global than ordinary Nim, because discarding one heap and repartitioning the rest changes the full state structure at once. We restrict the number of piles to 3, bound the third pile by $m \leq 5$, and obtain 6 test cases in total.

Entropy Reduction Game. A move transfers chips from one pile to another in such a way that the heaps become more nearly equal in size. Its characteristic feature is that a legal move is guided by reducing dispersion across heaps instead of simply decreasing one heap independently. We restrict the number of piles to 3, bound the third pile by $m \leq 5$, and include the unrestricted version, giving 7 test cases.

Min Game. There are two piles of chips v_1 and v_2, and a move removes k chips from the larger pile where k is bounded by the smaller pile. The strategy of this class depends directly on comparing the two heaps, because the legal amount that may be removed is determined by the current minimum. We

restrict the maximum of v_2 to be at most 5 and also include the unrestricted version, giving 6 test cases in total.

l-Slow Nim. The number of chips removed in one move is restricted to the interval $[1, l]$. This class imposes an explicit upper bound on each move and isolates the effect of bounded local decreases within the Nim setting. We restrict the number of piles to 3, bound the third pile by $m \leq 5$, let l range up to 30, and obtain 180 test cases.

Sharing Nim. In one turn, a player takes some chips from one pile and adds them to another pile, provided that after the move the destination pile does not exceed the source pile. This class emphasizes redistribution under balance constraints, and therefore tests interactions among heaps beyond pure deletion. We restrict the number of piles to at most 3, bound the third pile by $m \leq 5$, and include the unrestricted version, yielding 7 test cases.

Small Nim. A player may remove chips only from a smallest heap, or from any heap tied for the smallest size. Its main feature is a strong local-minimum restriction, so legal play is determined by the current order among heaps. We restrict the number of piles to at most 3, bound the third pile by $m \leq 5$, and include the unrestricted version, giving 7 test cases.

Greedy Nim. A player may remove chips only from a largest heap, or from any heap tied for the largest size. As the dual of Small Nim, this class imposes a maximal-heap preference and highlights how local extremal choices shape the strategy. We restrict the number of piles to at most 3, bound the third pile by $m \leq 5$, and include the unrestricted version, giving 7 test cases.

Odd-or-Even Nim. At the beginning, the first player chooses whether the second player must take an odd or an even number of chips. After each move, the player who made that move chooses the odd-or-even requirement for the opponent's next move. This class differs from standard Nim by adding a control variable, so legal actions depend on both heap sizes and the parity requirement chosen for the current move. We restrict the number of piles to at most 2, bound the second pile by $m \leq 5$, and include the unrestricted version, obtaining 7 test cases.

Modular Nim. A move removes k chips from one pile subject to a modular arithmetic condition $k \not\equiv_n x$. This class introduces arithmetic constraints directly into the move rule, so strategically relevant patterns can depend on residue classes instead of only on magnitudes. We restrict the number of piles to at most 3 and enumerate all residues $0 \leq x < n$, which gives 5 test cases.

Pointed Nim. A pointer indicates from which heap the next move must be made, and after removing chips the player must redirect the pointer to a different nonempty heap. The crucial feature of this class is the additional control component, because the next legal move depends on the current pointer as well as the heap sizes. We restrict the number of piles to at most 3, bound the third pile by $m \leq 5$, and include the unrestricted version, obtaining 7 test cases.

End Nim. The piles are ordered, and a player may remove chips only from the leftmost or rightmost nonempty pile. This class adds a positional restriction to Nim, because play is limited to boundary heaps rather than any heap in the sequence. We restrict the number of piles to at most 3, bound the third pile by $m \leq 5$, and include the unrestricted version, giving 7 test cases.

Loop End Nim. This variant keeps the end-move restriction of End Nim, but the game ends when only one pile remains nonempty. Its distinctive feature lies in the altered terminal condition, which changes the strategic role of reducing the position to a single surviving heap. We restrict the number of piles to at most 3, bound the third pile by $m \leq 5$, and include the unrestricted version, obtaining 7 test cases.

Staircase Nim. A staircase of n steps contains coins on some steps, and a move transfers any positive number of coins from step v_i to the next lower step v_{i-1}. This class differs from ordinary Nim because tokens move along adjacent positions instead of being removed directly from an isolated heap. We restrict the number of steps to at most 3, bound the third step by $m \leq 5$, and include the unrestricted version, giving 7 test cases.

Circular Nim. There are n piles arranged in a circle, and one move chooses m consecutive piles and removes at least one chip from one or more of them.

The defining feature of this class is circular adjacency, which makes the global layout of heaps part of the legal move relation. We consider the cases with $1 \leq n \leq 5$ and $0 \leq m \leq n$, which gives 15 test cases.

7.2.3. Welter games

This category includes 3 classes: Welter [38], l-Welter [47], and Max-Welter [48], contributing 21 test cases.

Welter There are at least 2 coins placed on distinct squares of a semi-infinite linear board and the squares numbered $0, 1, 2, \cdots$, where v_i denotes the position of the i-th coin. Two players alternately move a coin to a lower unoccupied square. The game ends when the player is unable to move any coin. We restrict the maximum number of coins to be 4, resulting in 3 test cases in this class.

l-Welter This is a generalization of Welter game. A player can move a coin by at most l squares from its current position. We restrict the maximum number of coins to be 4 and vary l from 1 to 5, yielding 15 test cases in total.

Max-Welter This is a variant of Welter game. A player can only move the coin in the largest square. We restrict the maximum number of coins to be 4, resulting in 3 test cases in this class.

7.2.4. Wythoff games

This category includes 2 classes: Wythoff [6], l-Wythoff [49], Monotonic l-Diet Wythoff [10], Wythoff with subtraction set S [50], Wythoff with a function f [9], 1st (l_1, l_2)-restricted Wythoff [51], 2nd (l_1, l_2)-restricted Wythoff [52], 3rd (l_1, l_2)-restricted Wythoff [53], and 4th (l_1, l_2)-restricted Wythoff [54], contributing $2,924$ test cases. (different restricted Wythoff??)

Wythoff There are exactly two piles of chips where v_1 and v_2 denotes the number of chips of the first and second pile, respectively. Each player

can take k_1 chips from the first pile or k_2 chips from the second pile as Nim game (namely, Nim move). She also can take the same number of chips from both piles (namely, Wythoff move). The game ends when both piles are empty. We consider the unrestricted configuration of v_2 and bounded configuration where $v_2 \leq m$ and $m \in [0, 5]$. In addition, following Liu and Zhao [9], we consider odd-odd, odd-even, even-odd and even-even restrictions on Wythoff game. Under the odd-odd restriction, the numbers k_1 and k_2 of chips removed from the first and second piles must be odd, respectively. Similarly, under the even-even restriction, both k_1 and k_2 are required to be even. The odd-even (resp. even-odd) restriction requires k_1 to be odd (resp. even) and k_2 even (resp. odd). In total, we obtain 35 test cases in this class.

l-Wythoff This is a generalization of Wythoff game where l is a positive integer. In a Wythoff move, each player take k_1 chips from the first pile and k_2 chips from the second pile s.t. $|k_1 - k_2| < l$. We let l range from 1 to 5, and consider the unrestricted and bounded configurations as the original Wythoff game. In addition, we impose the four additional restrictions on k_1 and k_2 as the original Wythoff on this class, yielding 175 test cases.

Monotonic l-Diet Wythoff This is a restriction of l-Wythoff game with an additional requirement where $v_1 \geq v_2$. We consider the range of l and the configurations as l-Wythoff. Since k_1 and k_2 must be the same in any Wythoff move in this class, we impose only odd-odd and even-even restrictions, resulting in 105 test cases.

Wythoff with subtraction set S Let S be a finite set of positive integers. Each player takes $k \in S$ chips from both two piles in a Wythoff move. We consider sets S of size at most 2 where each element of S ranges from 1 to 5. We consider the unrestricted and bounded configurations as the original Wythoff game. Since k_1 and k_2 must be the same in any Wythoff move in this class, we impose only odd-odd and even-even restrictions on k_1 and k_2, yielding 210 test cases.

Wythoff with a function f In the Wythoff move, player could take k_1 chips from the first pile and k_2 chips from the second one where k_1 and k_2 is restricted by the condition: $0 < k_1 \leq k_2 < f(k_1)$. We use $f(k) = k+i$ with $1 \leq i \leq 5$ as the function f. We consider the unrestricted and bounded configurations and impose the same restrictions as the original Wythoff game. In summary, this class contains a total of 175 test cases.

1st (l_1, l_2)-restricted Wythoff This is the first restriction of Wythoff game with two nonegative integers l_1 and l_2 s.t. $l_1 \leq l_2$. After a Wythoff move, the new state (v_1, v_2) must satisfy $\min(v_1, v_2) \geq l_1$ and $\max(v_1, v_2) \geq l_2$. We let both l_1 and l_2 range from 0 to 5, and consider the unrestricted and bounded configurations as the original Wythoff game. Since k_1 and k_2 must be the same in any Wythoff move in this class, we impose only odd-odd and even-even restrictions on this class, yielding 315 test cases.

2nd (l_1, l_2)-restricted Wythoff This is the second restriction of Wythoff game with two positive integers l_1 and l_2. In both Nim and Wythoff moves, each player takes k_1 chips from the first pile and k_2 chips from the second pile s.t. $\min(k_1, k_2) < l_1$ or $|k_1 - k_2| < l_2$. We let both l_1 and l_2 range from 1 to 5. We consider the unrestricted and bounded configurations and impose the same restrictions as the original Wythoff game. In summary, this class contains a total of 875 test cases.

3rd (l_1, l_2)-restricted Wythoff This is the third restriction of Wythoff game with two positive integers l_1 and l_2. In a Wythoff move, each player take k_1 chips from the first pile and k_2 chips from the second pile s.t. $0 \leq |k_2 - k_1| < (l_1 - 1)\lambda + l_2$ where $\lambda = \min(k_1, k_2)$. We impose the same restrictions on l_1, l_2, k_1 and k_2 and the unrestricted and bounded configurations as 2nd (l_1, l_2)-restricted Wythoff on this class, yielding 875 test cases.

4th (l_1, l_2)-restricted Wythoff This is the fourth restriction of Wythoff game with two positive integers l_1 and l_2. In a Wythoff move, each player take k_1

chips from the first pile and k_2 chips from the second pile s.t. $k_1 \cdot l_1 = k_2 \cdot l_2$. We impose the same restrictions on l_1, l_2, k_1 and k_2 and the unrestricted and bounded configurations as 2nd (l_1, l_2)-restricted Wythoff on this class, yielding 875 test cases.

7.2.5. Chomp games

This category contains three classes: Chomp [7], k-Diet Chomp [10], and L-shaped Chomp [7]. Together, they contribute 12 test cases.

Chomp Cookies are laid out on rows, where v_i denotes the number of cookies on row i. The number of cookies is non-decreasing, that is, if $i > j$, then $v_i \leq v_j$. At each move, a player has to eat a remaining cookie, together with all cookies located to the right and above it. The cookie at the top-left position $(1, 1)$ is poisoned, and the game ends once it is eaten. We include one two-rowed instance, five bounded three-rowed instances with $v_3 \leq m$ for $m \in \{1, \ldots, 5\}$, and one unbounded three-rowed instance. This gives 7 Chomp test cases.

k-Diet Chomp Diet Chomp restricts the number of cookies removed in one Chomp move [10]. For a fixed k, a legal move may remove at most k cookies in total; the paper calls this game k-Diet Chomp. For boards with two rows, we take $k = 2$ and $k = 3$; the same choices are used for boards with three rows. These four parameter combinations form the 4 test cases in this class.

L-shaped Chomp This is Chomp game with an L-shaped board with vertical and horizontal lines. The state variables v_1 and v_2 denote numbers of chips on the vertical and horizontal lines respectively. There is only 1 test case in this class.

7.3. Experimental Results

We compare the performance of SynEnum and SynMS under the normal and misère rules, respectively.

The solved-case counts in the tables are taken from the reports, while the running-time and size values are computed from the summary data after aligning the solved cases with the table counts.

The running-time curves are generated from the same filtered summary data.

In each table, the column "Solved" denotes the number of test cases solved by each approach.

The value in parentheses in a "Solved" cell denotes the number of test cases solved only by that approach.

The columns "Time" and "Size" denote the average running time and average solution size over the solved test cases for each approach.

For "Time" and "Size", the value in parentheses denotes the corresponding average over the test cases solved by both approaches.

The optimal data of each group are in bold.

7.3.1. Experimental Results under normal rule

Comparison on Subtraction game. Table 4 presents the results for synthesizing winning formulas in the Subtraction game under the normal rule. Out of 479 test cases, SynEnum solves 136 formula cases, while SynMS solves 467, with success rates of 28.4% and 97.5%, respectively. Thus SynMS solves 331 more formula cases than SynEnum. The average formula synthesis time is 15.61 seconds for SynEnum and 52.06 seconds for SynMS. Table 5 summarizes the corresponding results for synthesizing winning strategies. SynEnum solves 136 strategy cases, whereas SynMS solves 466 strategy cases. This gives SynMS an advantage of 330 strategy cases in this benchmark group. The average strategy synthesis time is 2.33 seconds for SynEnum and 14.54 seconds for SynMS. In terms of solution size, SynEnum produces smaller formulas and strategies on average, with average sizes 11.44 and 63.72, while SynMS has average sizes 33.05 and 118.25. Figure 2 shows the cumulative number of solved test cases over time using the same solved-case filtering as the tables.

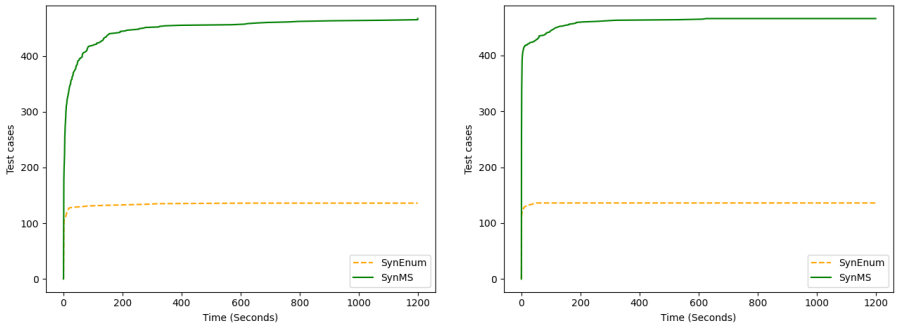
Comparison on Nim game. Table 6 presents the results for synthesizing winning formulas in the Nim game under the normal rule. Out of 302 test

Category	Test cases	Formula					
		Solved		Time		Size	
		SynEnum	SynMS	SynEnum	SynMS	SynEnum	SynMS
Take-away	30	**30**(0)	**30**(0)	6.77(6.77)	**6.11**(**6.11**)	**3.00**(**3.00**)	**3.00**(**3.00**)
Subtraction	385	98(0)	**382**(284)	**11.27**(**11.27**)	37.85(22.17)	**14.14**(**14.14**)	34.84(41.21)
S, D-Mark	64	8(0)	**55**(47)	**101.97**(101.97)	175.77(**3.44**)	**10.00**(**10.00**)	36.96(29.25)
Total	479	136(0)	**467**(331)	**15.61**(**15.61**)	52.06(17.52)	**11.44**(**11.44**)	33.05(32.08)

Table 4: Results of synthesizing winning formulas for Sub games under normal rule

Category	Test cases	Strategy					
		Solved		Time		Size	
		SynEnum	SynMS	SynEnum	SynMS	SynEnum	SynMS
Take-away	30	**30**(0)	**30**(0)	9.22(9.22)	**7.64**(**7.64**)	**93.00**(**93.00**)	**93.00**(**93.00**)
Subtraction	385	98(0)	**382**(284)	**0.38**(**0.38**)	0.61(0.66)	**55.63**(**55.63**)	121.80(125.14)
S, D-Mark	64	8(0)	**54**(46)	**0.46**(**0.46**)	116.93(1.95)	**53.00**(**53.00**)	106.98(202.25)
Total	479	136(0)	**466**(330)	**2.33**(2.33)	14.54(**2.28**)	**63.72**(**63.72**)	118.25(122.59)

Table 5: Results of synthesizing winning strategies for Sub games under normal rule



(a) Cumulative solved formula cases

(b) Cumulative solved strategy cases

Figure 2: Cumulative solved test cases over time for Subtraction game under normal rule

cases, SynEnum solves 62 formula cases, while SynMS solves 95, with success rates of 20.5% and 31.5%, respectively. Thus SynMS solves 33 more formula cases than SynEnum. The average formula synthesis time is 9.77 seconds for SynEnum and 5.70 seconds for SynMS. Table 7 summarizes the corresponding

results for synthesizing winning strategies. SynEnum solves 59 strategy cases, whereas SynMS solves 86 strategy cases. This gives SynMS an advantage of 27 strategy cases in this benchmark group. The average strategy synthesis time is 44.60 seconds for SynEnum and 63.38 seconds for SynMS. In terms of solution size, SynEnum produces smaller formulas and strategies on average, with average sizes 5.77 and 28.63, while SynMS has average sizes 71.34 and 481.79. Figure 3 shows the cumulative number of solved test cases over time using the same solved-case filtering as the tables.

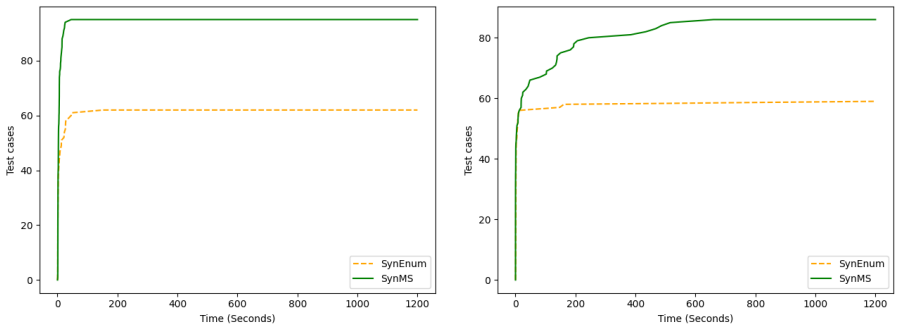
Category	Test cases	Formula					
		Solved		Time		Size	
		SynEnum	SynMS	SynEnum	SynMS	SynEnum	SynMS
Nim	7	1(0)	**2**(1)	**1.13**(1.13)	3.66(**1.12**)	**4.00**(4.00)	23.50(**3.00**)
Monotonic Nim	7	**7**(0)	**7**(0)	**0.94**(**0.94**)	1.28(1.28)	4.86(4.86)	**3.86**(**3.86**)
Empty and Transfer	7	**2**(1)	1(0)	**2.83**(4.50)	2.98(**2.98**)	**7.50**(**7.00**)	18.00(18.00)
Empty All But One	6	**6**(0)	**6**(0)	24.40(24.40)	**4.64**(**4.64**)	**5.33**(**5.33**)	32.00(32.00)
Empty and Redistribute	6	1(1)	**2**(2)	36.61(-)	**2.65**(-)	**13.00**(-)	178.00(-)
Entropy Reduction Game	7	**1**(0)	**1**(0)	**12.63**(**12.63**)	19.53(19.53)	**8.00**(**8.00**)	12.00(12.00)
Min Game	6	1(0)	**3**(2)	**3.10**(3.10)	8.95(**3.00**)	**6.00**(6.00)	117.67(**5.00**)
l-Slow Nim	180	9(1)	**11**(3)	**1.42**(**1.40**)	7.19(6.46)	**4.89**(**4.88**)	135.09(59.50)
Sharing Nim	7	4(0)	**6**(2)	**4.78**(4.78)	6.31(**3.83**)	**5.00**(**5.00**)	10.83(5.25)
Small Nim	7	**7**(0)	**7**(0)	18.29(18.29)	**1.70**(**1.70**)	**7.29**(**7.29**)	11.86(11.86)
Greedy Nim	7	2(0)	**7**(5)	6.92(6.92)	**5.47**(**1.09**)	**8.50**(**8.50**)	159.86(19.50)
Odd-or-Even Nim	7	2(0)	**6**(4)	3.88(3.88)	**3.28**(**1.34**)	**7.00**(**7.00**)	230.83(21.00)
Modular Nim	5	1(0)	**2**(1)	**1.12**(**1.12**)	1.94(1.32)	**4.00**(**4.00**)	30.00(12.00)
Pointed Nim	7	1(0)	**7**(6)	23.07(**23.07**)	**9.84**(24.00)	**4.00**(**4.00**)	49.29(**4.00**)
End Nim	7	1(0)	**7**(6)	**1.08**(1.08)	2.48(**0.86**)	**4.00**(**4.00**)	107.71(**4.00**)
Loop End Nim	7	2(0)	**7**(5)	78.32(78.32)	**13.02**(**6.46**)	**9.50**(**9.50**)	51.86(30.00)
Staircase Nim	7	**7**(0)	**7**(0)	**1.79**(**1.79**)	2.52(2.52)	3.71(3.71)	**2.86**(**2.86**)
Circular Nim	15	**7**(1)	6(0)	**2.52**(**1.65**)	9.58(9.58)	**6.14**(**5.67**)	15.33(15.33)
Total	302	62(4)	**95**(37)	9.77(9.63)	**5.70**(**4.51**)	**5.77**(**5.57**)	71.34(19.14)

Table 6: Results of synthesizing winning formulas for Nim games under normal rule

Comparison on Welter game. Table 8 presents the results for synthesizing winning formulas in the Welter game under the normal rule. Out of 7 test cases, SynEnum solves 4 formula cases, while SynMS solves 5, with success rates of 57.1% and 71.4%, respectively. Thus SynMS solves 1 more formula

Category	Test cases	Strategy					
		Solved		Time		Size	
		SynEnum	SynMS	SynEnum	SynMS	SynEnum	SynMS
Nim	7	1(0)	**2**(1)	1.12(1.12)	**0.17**(**0.04**)	**12.00**(**12.00**)	54.00(**12.00**)
Monotonic Nim	7	**7**(0)	**7**(0)	1.16(1.16)	**0.09**(**0.09**)	28.86(28.86)	**22.86**(**22.86**)
Empty and Transfer	7	**2**(1)	1(0)	**0.78**(**0.71**)	9.61(9.61)	18.00(14.00)	**12.00**(**12.00**)
Empty All But One	6	**6**(0)	**6**(0)	**7.47**(**7.47**)	16.22(16.22)	**5.33**(**5.33**)	**5.33**(**5.33**)
Empty and Redistribute	6	0(0)	**1**(1)	-(-)	24.35(-)	-(-)	9.00(-)
Entropy Reduction Game	7	0(0)	**1**(1)	-(-)	4.08(-)	-(-)	112.00(-)
Min Game	6	1(0)	**3**(2)	**0.46**(**0.46**)	45.96(2.32)	**11.00**(**11.00**)	3216.33(113.00)
l-Slow Nim	180	9(1)	**11**(3)	**0.49**(**0.41**)	38.37(51.97)	**30.89**(**26.75**)	821.09(311.25)
Sharing Nim	7	3(0)	**6**(3)	**0.44**(**0.44**)	35.95(22.21)	**22.33**(**22.33**)	1234.50(627.33)
Small Nim	7	**7**(0)	**7**(0)	2.72(2.72)	**1.25**(**1.25**)	72.57(72.57)	**30.86**(**30.86**)
Greedy Nim	7	2(0)	**5**(3)	**1.49**(**1.49**)	282.73(21.04)	**49.50**(**49.50**)	1101.40(177.50)
Odd-or-Even Nim	7	2(0)	**6**(4)	**0.89**(**0.89**)	259.60(19.98)	**16.50**(**16.50**)	859.83(66.00)
Modular Nim	5	1(0)	**2**(1)	**0.51**(**0.51**)	2.15(3.63)	**14.00**(**14.00**)	55.00(32.00)
Pointed Nim	7	1(0)	**7**(6)	0.74(0.74)	**0.53**(**0.05**)	**8.00**(**8.00**)	11.43(**8.00**)
End Nim	7	1(0)	**4**(3)	**1.08**(1.08)	147.56(**0.04**)	**8.00**(**8.00**)	336.75(**8.00**)
Loop End Nim	7	2(0)	**7**(5)	**1.24**(**1.24**)	109.37(40.79)	**25.00**(**25.00**)	335.86(467.50)
Staircase Nim	7	**7**(0)	**7**(0)	2.06(2.06)	**0.07**(**0.07**)	23.71(23.71)	**17.43**(**17.43**)
Circular Nim	15	**7**(4)	3(0)	360.97(**0.94**)	**64.87**(64.87)	23.57(**7.00**)	**7.00**(**7.00**)
Total	302	59(6)	**86**(33)	**44.60**(**1.99**)	63.38(18.18)	**28.63**(**27.53**)	481.79(123.21)

Table 7: Results of synthesizing winning strategies for Nim games under normal rule



(a) Cumulative solved formula cases

(b) Cumulative solved strategy cases

Figure 3: Cumulative solved test cases over time for Nim game under normal rule

cases than SynEnum. The average formula synthesis time is 62.06 seconds for SynEnum and 3.50 seconds for SynMS. Table 9 summarizes the corresponding results for synthesizing winning strategies. SynEnum solves 4 strategy cases,

whereas SynMS solves 5 strategy cases. This gives SynMS an advantage of 1 strategy cases in this benchmark group. The average strategy synthesis time is 1.12 seconds for SynEnum and 0.55 seconds for SynMS. In terms of solution size, SynEnum produces smaller formulas and strategies on average, with average sizes 8.00 and 39.00, while SynMS has average sizes 74.40 and 399.20. Figure 4 shows the cumulative number of solved test cases over time using the same solved-case filtering as the tables.

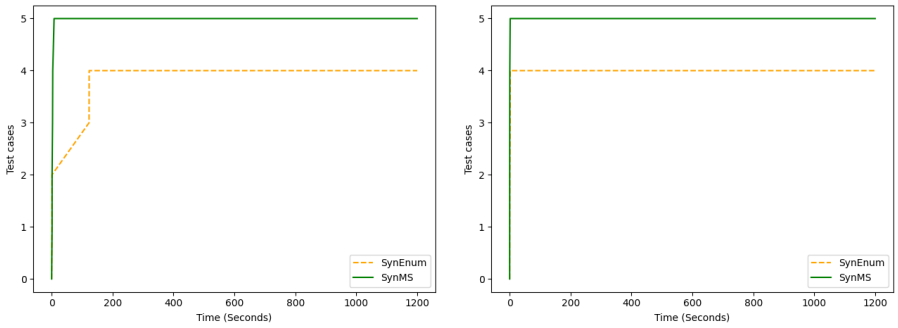
Category	Test cases	Formula					
		Solved		Time		Size	
		SynEnum	SynMS	SynEnum	SynMS	SynEnum	SynMS
Welter	1	**1**(0)	**1**(0)	123.22(123.22)	**3.00**(**3.00**)	**12.00**(**12.00**)	36.00(36.00)
l-Welter	5	2(0)	**3**(1)	**1.09**(**1.09**)	4.35(2.53)	**4.00**(**4.00**)	100.00(42.00)
Max-Welter	1	**1**(0)	**1**(0)	122.86(122.86)	**1.46**(**1.46**)	**12.00**(**12.00**)	36.00(36.00)
Total	7	4(0)	**5**(1)	62.06(62.06)	**3.50**(**2.38**)	**8.00**(**8.00**)	74.40(39.00)

Table 8: Results of synthesizing winning formulas for Welter games under normal rule

Category	Test cases	Strategy					
		Solved		Time		Size	
		SynEnum	SynMS	SynEnum	SynMS	SynEnum	SynMS
Welter	1	**1**(0)	**1**(0)	1.75(1.75)	**0.23**(**0.23**)	44.00(44.00)	**38.00**(**38.00**)
l-Welter	5	2(0)	**3**(1)	**0.39**(0.39)	0.78(**0.30**)	**30.00**(**30.00**)	637.33(197.00)
Max-Welter	1	**1**(0)	**1**(0)	1.96(1.96)	**0.17**(**0.17**)	52.00(52.00)	**46.00**(**46.00**)
Total	7	4(0)	**5**(1)	1.12(1.12)	**0.55**(**0.25**)	**39.00**(**39.00**)	399.20(119.50)

Table 9: Results of synthesizing winning strategies for Welter games under normal rule

Comparison on Wythoff game. Table 10 presents the results for synthesizing winning formulas in the Wythoff game under the normal rule. Out of 2924 test cases, SynEnum solves 1863 formula cases, while SynMS solves 2679, with success rates of 63.7% and 91.6%, respectively. Thus SynMS solves 816 more formula cases than SynEnum. The average formula synthesis time is 24.24 seconds for SynEnum and 15.29 seconds for SynMS. Table 11 summarizes the corresponding results for synthesizing winning strategies. SynEnum solves 1815



(a) Cumulative solved formula cases

(b) Cumulative solved strategy cases

Figure 4: Cumulative solved test cases over time for Welter game under normal rule

strategy cases, whereas SynMS solves 2200 strategy cases. This gives SynMS an advantage of 385 strategy cases in this benchmark group. The average strategy synthesis time is 2.08 seconds for SynEnum and 246.62 seconds for SynMS. In terms of solution size, SynEnum produces smaller formulas and strategies on average, with average sizes 5.57 and 37.95, while SynMS has average sizes 44.80 and 380.59. Figure 5 shows the cumulative number of solved test cases over time using the same solved-case filtering as the tables.

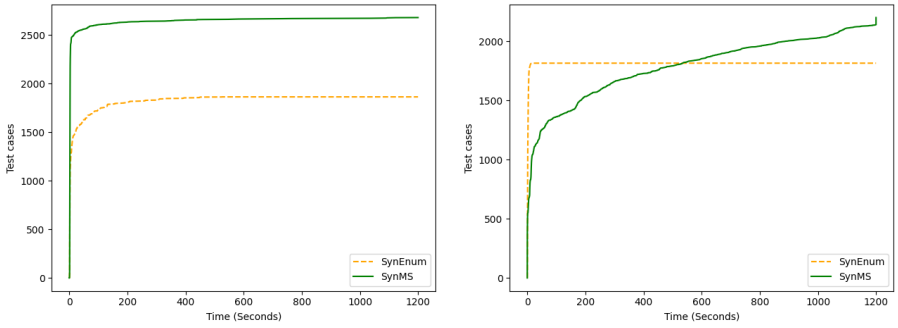
Category	Test cases	Formula					
		Solved		Time		Size	
		SynEnum	SynMS	SynEnum	SynMS	SynEnum	SynMS
Wythoff	21	13(0)	**19**(6)	11.18(11.18)	**2.44**(**2.56**)	**5.77**(**5.77**)	49.95(17.54)
l-Wythoff	140	93(1)	**120**(28)	**25.64**(25.81)	40.09(**21.19**)	**6.45**(**6.43**)	63.65(41.23)
Monotonic l-Diet Wythoff	105	81(1)	**103**(23)	13.69(13.81)	**6.39**(**3.14**)	**5.49**(**5.49**)	78.92(40.67)
Wythoff with subtraction set S	315	177(0)	**287**(110)	22.86(22.86)	**2.84**(**2.49**)	**5.76**(**5.76**)	55.32(20.49)
Wythoff with a function f	168	118(0)	**154**(36)	16.44(16.44)	**4.71**(**2.97**)	**5.59**(**5.59**)	47.68(29.14)
1st(l_1,l_2)restricted Wythoff	315	119(0)	**285**(166)	25.64(25.64)	**2.26**(**2.05**)	**6.03**(**6.03**)	57.56(21.94)
2nd(l_1,l_2)restricted Wythoff	600	543(0)	**600**(57)	29.32(29.32)	**2.12**(**2.05**)	**4.83**(**4.83**)	24.04(17.27)
3rd(l_1,l_2)restricted Wythoff	700	392(0)	**578**(186)	**29.69**(29.69)	50.54(**9.08**)	**6.51**(**6.51**)	50.73(31.17)
4th(l_1,l_2)restricted Wythoff	560	327(0)	**533**(206)	15.06(15.06)	**5.21**(**2.29**)	**5.18**(**5.18**)	37.42(13.22)
Total	2924	1863(2)	**2679**(818)	24.24(24.26)	**15.29**(**4.67**)	**5.57**(**5.57**)	44.80(23.03)

Table 10: Results of synthesizing winning formulas for Wythoff games under normal rule

Comparison on Chomp game. Table 12 presents the results for synthesizing winning formulas in the Chomp game under the normal rule. Out of 8 test cases, SynEnum solves 2 formula cases, while SynMS solves 3, with success

Category	Test cases	Strategy					
		Solved		Time		Size	
		SynEnum	SynMS	SynEnum	SynMS	SynEnum	SynMS
Wythoff	21	**13**(1)	12(0)	**1.22**(**1.05**)	6.48(6.48)	**36.15**(**27.00**)	55.75(55.75)
l-Wythoff	140	**93**(24)	73(4)	**2.77**(**2.44**)	402.79(376.59)	**56.42**(**45.86**)	434.71(298.39)
Monotonic l-Diet Wythoff	105	81(1)	**103**(23)	**0.47**(0.46)	0.64(**0.39**)	**27.43**(**27.05**)	477.36(245.06)
Wythoff with subtraction set S	315	177(0)	**253**(76)	**1.50**(**1.50**)	15.14(6.17)	**31.39**(**31.39**)	437.05(159.77)
Wythoff with a function f	168	**118**(19)	110(11)	**1.92**(**1.79**)	178.94(150.36)	**35.04**(**29.24**)	216.99(170.13)
1st(l_1,l_2)restricted Wythoff	315	119(0)	**257**(138)	**1.76**(**1.76**)	11.06(5.66)	**31.39**(**31.39**)	457.32(207.94)
2nd(l_1,l_2)restricted Wythoff	600	543(15)	**577**(49)	**1.70**(**1.66**)	542.39(488.22)	**34.10**(**32.30**)	400.03(214.77)
3rd(l_1,l_2)restricted Wythoff	700	**392**(54)	350(12)	**3.47**(**3.06**)	429.75(408.42)	**57.16**(**50.30**)	313.33(266.46)
4th(l_1,l_2)restricted Wythoff	560	279(0)	**465**(186)	**1.72**(**1.72**)	50.08(30.76)	**23.65**(**23.65**)	351.11(230.91)
Total	2924	1815(114)	**2200**(499)	**2.08**(**1.92**)	246.62(262.88)	**37.95**(**34.39**)	380.59(222.58)

Table 11: Results of synthesizing winning strategies for Wythoff games under normal rule



(a) Cumulative solved formula cases

(b) Cumulative solved strategy cases

Figure 5: Cumulative solved test cases over time for Wythoff game under normal rule

rates of 25.0% and 37.5%, respectively. Thus SynMS solves 1 more formula cases than SynEnum. The average formula synthesis time is 1.40 seconds for SynEnum and 4.46 seconds for SynMS. Table 13 summarizes the corresponding results for synthesizing winning strategies. SynEnum solves 2 strategy cases, whereas SynMS solves 2 strategy cases. Both approaches solve 2 strategy cases in this benchmark group. The average strategy synthesis time is 2.26 seconds for SynEnum and 5.26 seconds for SynMS. In terms of solution size, SynEnum produces smaller formulas and strategies on average, with average sizes 4.00 and 9.50, while SynMS has average sizes 42.00 and 10.00. Figure 6 shows the cumulative number of solved test cases over time using the same solved-case

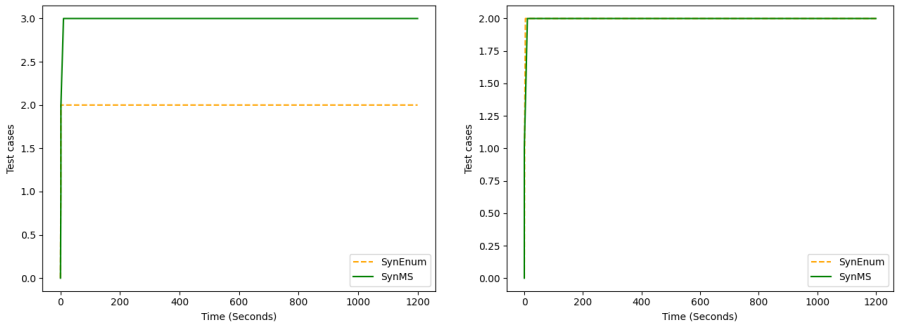
filtering as the tables.

Category	Test cases	Formula					
		Solved		Time		Size	
		SynEnum	SynMS	SynEnum	SynMS	SynEnum	SynMS
Chomp	7	1(0)	**2**(1)	**1.21**(**1.21**)	5.93(1.64)	**4.00**(4.00)	61.50(**3.00**)
L-shaped Chomp	1	**1**(0)	**1**(0)	1.59(1.59)	**1.53**(**1.53**)	4.00(4.00)	**3.00**(**3.00**)
Total	8	2(0)	**3**(1)	**1.40**(**1.40**)	4.46(1.59)	**4.00**(4.00)	42.00(**3.00**)

Table 12: Results of synthesizing winning formulas for Chomp games under normal rule

Category	Test cases	Strategy					
		Solved		Time		Size	
		SynEnum	SynMS	SynEnum	SynMS	SynEnum	SynMS
Chomp	7	**1**(0)	**1**(0)	**3.57**(**3.57**)	10.47(10.47)	**9.00**(**9.00**)	10.00(10.00)
L-shaped Chomp	1	**1**(0)	**1**(0)	0.96(0.96)	**0.04**(**0.04**)	**10.00**(**10.00**)	**10.00**(**10.00**)
Total	8	**2**(0)	**2**(0)	**2.26**(**2.26**)	5.26(5.26)	**9.50**(**9.50**)	10.00(10.00)

Table 13: Results of synthesizing winning strategies for Chomp games under normal rule



(a) Cumulative solved formula cases

(b) Cumulative solved strategy cases

Figure 6: Cumulative solved test cases over time for Chomp game under normal rule

7.3.2. Experimental Results under misère rule

Comparison on Subtraction game. Table 14 presents the results for synthesizing winning formulas in the Subtraction game under the misère rule.

Out of 479 test cases, SynEnum solves 178 formula cases, while SynMS solves 466, with success rates of 37.2% and 97.3%, respectively. Thus SynMS solves 288 more formula cases than SynEnum. The average formula synthesis time is 16.17 seconds for SynEnum and 17.91 seconds for SynMS. Table 15 summarizes the corresponding results for synthesizing winning strategies. SynEnum solves 176 strategy cases, whereas SynMS solves 466 strategy cases. This gives SynMS an advantage of 290 strategy cases in this benchmark group. The average strategy synthesis time is 2.17 seconds for SynEnum and 3.62 seconds for SynMS. In terms of solution size, SynEnum produces smaller formulas and strategies on average, with average sizes 9.13 and 55.60, while SynMS has average sizes 35.81 and 133.66. Figure 7 shows the cumulative number of solved test cases over time using the same solved-case filtering as the tables.

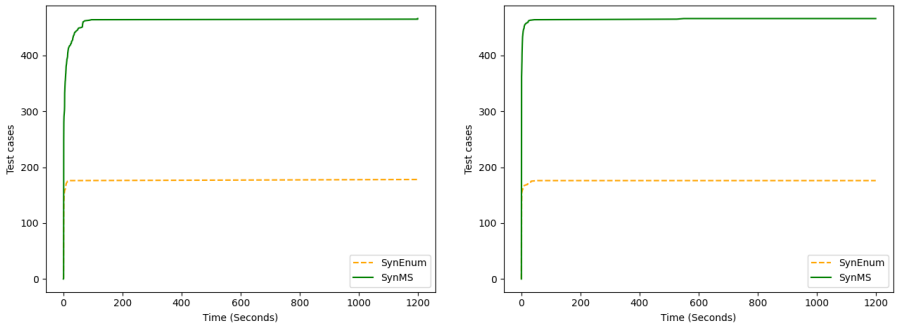
Category	Test cases	Formula					
		Solved		Time		Size	
		SynEnum	SynMS	SynEnum	SynMS	SynEnum	SynMS
Take-away	30	**30**(0)	**30**(0)	**5.13**(**5.13**)	5.82(5.82)	**3.00**(**3.00**)	**3.00**(**3.00**)
Subtraction	385	96(0)	**382**(286)	27.98(27.98)	**8.40**(**8.05**)	**14.38**(**14.38**)	41.63(22.49)
S, D-Mark	64	52(0)	**54**(2)	**0.74**(**0.74**)	91.90(1.04)	**3.00**(**3.00**)	12.83(**3.00**)
Total	479	178(0)	**466**(288)	**16.17**(16.17)	17.91(**5.63**)	**9.13**(**9.13**)	35.81(13.51)

Table 14: Results of synthesizing winning formulas for Sub games under misère rule

Category	Test cases	Strategy					
		Solved		Time		Size	
		SynEnum	SynMS	SynEnum	SynMS	SynEnum	SynMS
Take-away	30	**30**(0)	**30**(0)	10.64(10.64)	**8.14**(**8.14**)	**93.00**(**93.00**)	**93.00**(**93.00**)
Subtraction	385	94(0)	**382**(288)	**0.50**(**0.50**)	0.94(0.56)	**58.34**(**58.34**)	149.31(83.90)
S, D-Mark	64	52(0)	**54**(2)	**0.32**(0.32)	20.08(**0.16**)	**29.08**(**29.08**)	45.52(**29.08**)
Total	479	176(0)	**466**(290)	**2.17**(2.17)	3.62(**1.73**)	**55.60**(**55.60**)	133.66(69.26)

Table 15: Results of synthesizing winning strategies for Sub games under misère rule

Comparison on Nim game. Table 16 presents the results for synthesizing



(a) Cumulative solved formula cases

(b) Cumulative solved strategy cases

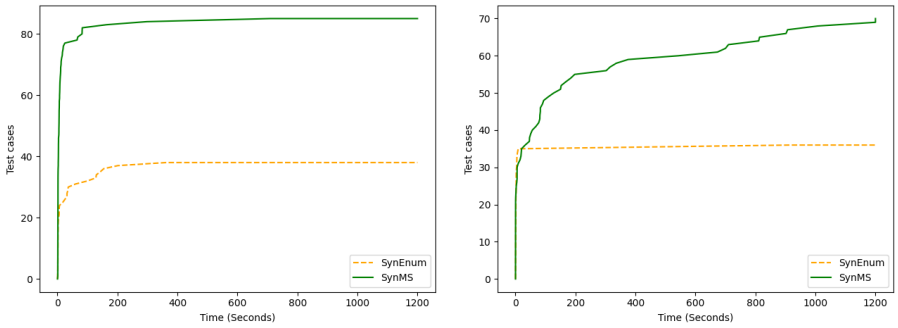
Figure 7: Cumulative solved test cases over time for Subtraction game under misère rule

winning formulas in the Nim game under the misère rule. Out of 302 test cases, SynEnum solves 38 formula cases, while SynMS solves 85, with success rates of 12.6% and 28.1%, respectively. Thus SynMS solves 47 more formula cases than SynEnum. The average formula synthesis time is 39.69 seconds for SynEnum and 22.51 seconds for SynMS. Table 17 summarizes the corresponding results for synthesizing winning strategies. SynEnum solves 36 strategy cases, whereas SynMS solves 70 strategy cases. This gives SynMS an advantage of 34 strategy cases in this benchmark group. The average strategy synthesis time is 27.04 seconds for SynEnum and 203.15 seconds for SynMS. In terms of solution size, SynEnum produces smaller formulas and strategies on average, with average sizes 7.05 and 28.31, while SynMS has average sizes 165.26 and 2069.77. Figure 8 shows the cumulative number of solved test cases over time using the same solved-case filtering as the tables.

Comparison on Welter game. Table 18 presents the results for synthesizing winning formulas in the Welter game under the misère rule. Out of 7 test cases, SynEnum solves 2 formula cases, while SynMS solves 7, with success rates of 28.6% and 100.0%, respectively. Thus SynMS solves 5 more formula cases than SynEnum. The average formula synthesis time is 1.62 seconds for SynEnum and 158.34 seconds for SynMS. Table 19 summarizes the corresponding results for synthesizing winning strategies. SynEnum solves 2 strategy cases,

Category	Test cases	Formula					
		Solved		Time		Size	
		SynEnum	SynMS	SynEnum	SynMS	SynEnum	SynMS
Nim	7	**1**(0)	**1**(0)	5.51(5.51)	**1.36**(**1.36**)	**9.00**(**9.00**)	28.00(28.00)
Monotonic Nim	7	4(1)	**5**(2)	126.91(121.58)	**77.54**(**2.48**)	**8.00**(**6.67**)	26.80(20.33)
Empty and Transfer	7	0(0)	**1**(1)	-(-)	2.46(-)	-(-)	70.00(-)
Empty All But One	6	0(0)	0(0)	-(-)	-(-)	-(-)	-(-)
Empty and Redistribute	6	0(0)	**2**(2)	-(-)	1.58(-)	-(-)	132.00(-)
Entropy Reduction Game	7	0(0)	**1**(1)	-(-)	13.56(-)	-(-)	36.00(-)
Min Game	6	1(0)	**4**(3)	**3.42**(3.42)	46.58(**3.07**)	**6.00**(**6.00**)	249.25(15.00)
l-Slow Nim	180	7(0)	**15**(8)	**1.36**(**1.36**)	70.67(5.37)	**4.71**(**4.71**)	190.73(37.14)
Sharing Nim	7	**4**(1)	**4**(1)	11.22(14.75)	**7.48**(**5.92**)	**6.50**(**8.67**)	27.00(29.33)
Small Nim	7	2(0)	**7**(5)	92.79(92.79)	**1.65**(**0.98**)	13.50(13.50)	**11.43**(**10.00**)
Greedy Nim	7	2(0)	**7**(5)	103.30(103.30)	**3.34**(**1.07**)	**11.00**(**11.00**)	336.43(49.00)
Odd-or-Even Nim	7	2(0)	**5**(3)	18.96(18.96)	**1.05**(**0.92**)	**8.50**(**8.50**)	377.60(66.00)
Modular Nim	5	1(0)	**2**(1)	**1.25**(**1.25**)	1.53(1.52)	**4.00**(**4.00**)	118.00(12.00)
Pointed Nim	7	0(0)	**4**(4)	-(-)	11.49(-)	-(-)	429.50(-)
End Nim	7	1(0)	**7**(6)	**2.74**(2.74)	3.61(**1.01**)	**8.00**(**8.00**)	295.00(**8.00**)
Loop End Nim	7	1(0)	**7**(6)	34.66(34.66)	**9.10**(**5.81**)	**13.00**(**13.00**)	113.57(45.00)
Staircase Nim	7	**7**(0)	**7**(0)	64.56(64.56)	**2.56**(**2.56**)	**6.86**(**6.86**)	23.00(23.00)
Circular Nim	15	5(0)	**6**(1)	**3.30**(**3.30**)	5.41(5.93)	**4.60**(**4.60**)	41.83(26.00)
Total	302	38(2)	**85**(49)	39.69(37.91)	**22.51**(**3.59**)	**7.05**(**7.11**)	165.26(29.39)

Table 16: Results of synthesizing winning formulas for Nim games under misère rule



(a) Cumulative solved formula cases

(b) Cumulative solved strategy cases

Figure 8: Cumulative solved test cases over time for Nim game under misère rule

whereas SynMS solves 5 strategy cases. This gives SynMS an advantage of 3 strategy cases in this benchmark group. The average strategy synthesis time

Category	Test cases	Strategy					
		Solved		Time		Size	
		SynEnum	SynMS	SynEnum	SynMS	SynEnum	SynMS
Nim	7	**1**(0)	**1**(0)	**2.08**(**2.08**)	5.13(5.13)	**26.00**(**26.00**)	138.00(138.00)
Monotonic Nim	7	4(1)	**5**(2)	**2.02**(**1.40**)	33.12(29.32)	**34.75**(**26.67**)	642.40(605.33)
Empty and Transfer	7	0(0)	**1**(1)	-(-)	315.57(-)	-(-)	52.00(-)
Empty All But One	6	0(0)	0(0)	-(-)	-(-)	-(-)	-(-)
Empty and Redistribute	6	0(0)	0(0)	-(-)	-(-)	-(-)	-(-)
Entropy Reduction Game	7	0(0)	0(0)	-(-)	-(-)	-(-)	-(-)
Min Game	6	1(0)	**4**(3)	**0.49**(**0.49**)	66.23(1.74)	**11.00**(**11.00**)	1967.00(239.00)
l-Slow Nim	180	7(0)	**9**(2)	**0.37**(0.37)	0.57(**0.19**)	**22.00**(**22.00**)	621.78(141.43)
Sharing Nim	7	**3**(2)	2(1)	**0.24**(**0.71**)	16.47(14.91)	**11.33**(**34.00**)	958.00(960.00)
Small Nim	7	2(0)	**7**(5)	1.40(**1.40**)	**0.85**(2.62)	**29.50**(**29.50**)	30.00(30.00)
Greedy Nim	7	2(0)	**6**(4)	**2.01**(**2.01**)	474.69(43.12)	**47.00**(**47.00**)	3101.17(1963.50)
Odd-or-Even Nim	7	2(0)	**5**(3)	**0.66**(**0.66**)	190.30(20.46)	**22.50**(**22.50**)	1859.20(551.50)
Modular Nim	5	1(0)	**2**(1)	**0.54**(**0.54**)	25.18(4.17)	**14.00**(**14.00**)	267.00(32.00)
Pointed Nim	7	0(0)	**4**(4)	-(-)	1004.20(-)	-(-)	616.75(-)
End Nim	7	1(0)	**7**(6)	**1.25**(1.25)	400.59(**0.05**)	**12.00**(**12.00**)	12199.00(**12.00**)
Loop End Nim	7	1(0)	**6**(5)	**1.15**(1.15)	181.08(**0.36**)	**16.00**(**16.00**)	1215.33(18.00)
Staircase Nim	7	**7**(0)	**7**(0)	**3.39**(**3.39**)	44.26(44.26)	**49.00**(**49.00**)	283.43(283.43)
Circular Nim	15	**4**(0)	**4**(0)	**231.20**(**231.20**)	339.47(339.47)	**18.00**(**18.00**)	79.75(79.75)
Total	302	36(3)	**70**(37)	**27.04**(**29.38**)	203.15(58.05)	**28.31**(**29.09**)	2069.77(351.45)

Table 17: Results of synthesizing winning strategies for Nim games under misère rule

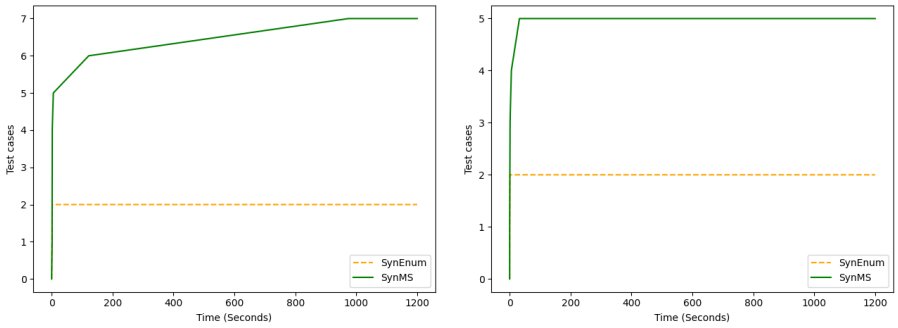
is 0.65 seconds for SynEnum and 7.97 seconds for SynMS. In terms of solution size, SynEnum produces smaller formulas and strategies on average, with average sizes 4.50 and 43.00, while SynMS has average sizes 263.14 and 535.60. Figure 9 shows the cumulative number of solved test cases over time using the same solved-case filtering as the tables.

Category	Test cases	Formula					
		Solved		Time		Size	
		SynEnum	SynMS	SynEnum	SynMS	SynEnum	SynMS
Welter	1	0(0)	**1**(1)	-(-)	1.89(-)	-(-)	570.00(-)
l-Welter	5	2(0)	**5**(3)	**1.62**(**1.62**)	221.01(3.53)	**4.50**(**4.50**)	240.00(114.00)
Max-Welter	1	0(0)	**1**(1)	-(-)	1.48(-)	-(-)	72.00(-)
Total	7	2(0)	**7**(5)	**1.62**(**1.62**)	158.34(3.53)	**4.50**(**4.50**)	263.14(114.00)

Table 18: Results of synthesizing winning formulas for Welter games under misère rule

Category	Test cases	Strategy					
		Solved		Time		Size	
		SynEnum	SynMS	SynEnum	SynMS	SynEnum	SynMS
Welter	1	0(0)	**1**(1)	-(-)	32.12(-)	-(-)	190.00(-)
l-Welter	5	2(0)	**3**(1)	**0.65**(**0.65**)	0.73(0.74)	**43.00**(**43.00**)	817.33(784.00)
Max-Welter	1	0(0)	**1**(1)	-(-)	5.57(-)	-(-)	36.00(-)
Total	7	2(0)	**5**(3)	**0.65**(**0.65**)	7.97(0.74)	**43.00**(**43.00**)	535.60(784.00)

Table 19: Results of synthesizing winning strategies for Welter games under misère rule



(a) Cumulative solved formula cases

(b) Cumulative solved strategy cases

Figure 9: Cumulative solved test cases over time for Welter game under misère rule

Comparison on Wythoff game. Table 20 presents the results for synthesizing winning formulas in the Wythoff game under the misère rule. Out of 2924 test cases, SynEnum solves 1933 formula cases, while SynMS solves 2691, with success rates of 66.1% and 92.0%, respectively. Thus SynMS solves 758 more formula cases than SynEnum. The average formula synthesis time is 37.54 seconds for SynEnum and 27.52 seconds for SynMS. Table 21 summarizes the corresponding results for synthesizing winning strategies. SynEnum solves 1919 strategy cases, whereas SynMS solves 2420 strategy cases. This gives SynMS an advantage of 501 strategy cases in this benchmark group. The average strategy synthesis time is 18.84 seconds for SynEnum and 386.13 seconds for SynMS. In terms of solution size, SynEnum produces smaller formulas and strategies on average, with average sizes 6.73 and 42.58, while SynMS has average sizes 72.72

and 453.61. Figure 10 shows the cumulative number of solved test cases over time using the same solved-case filtering as the tables.

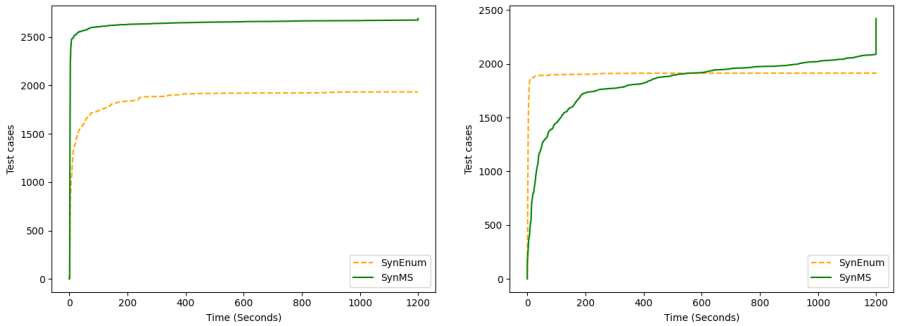
Category	Test cases	Formula					
		Solved		Time		Size	
		SynEnum	SynMS	SynEnum	SynMS	SynEnum	SynMS
Wythoff	21	10(0)	**19**(9)	41.51(41.51)	**2.57**(**2.89**)	**6.60**(**6.60**)	152.95(35.30)
l-Wythoff	140	80(0)	**123**(43)	48.85(48.85)	**46.97**(**2.95**)	**7.21**(**7.21**)	75.62(45.55)
Monotonic l-Diet Wythoff	105	61(0)	**104**(43)	31.07(31.07)	**11.34**(**2.98**)	**5.28**(**5.28**)	66.65(15.21)
Wythoff with subtraction set S	315	200(0)	**287**(87)	24.59(24.59)	**4.75**(**2.62**)	**6.28**(**6.28**)	106.17(38.67)
Wythoff with a function f	168	116(0)	**153**(37)	38.93(38.93)	**6.13**(**3.23**)	**7.37**(**7.37**)	63.46(39.89)
1st(l_1,l_2)restricted Wythoff	315	214(0)	**285**(71)	26.60(26.60)	**2.26**(**2.37**)	**6.83**(**6.83**)	99.46(48.95)
2nd(l_1,l_2)restricted Wythoff	600	440(0)	**600**(160)	37.98(37.98)	**2.80**(**2.39**)	**6.36**(**6.36**)	59.01(34.23)
3rd(l_1,l_2)restricted Wythoff	700	404(1)	**588**(185)	**55.56**(55.67)	96.15(**4.50**)	**7.18**(**7.17**)	54.75(44.65)
4th(l_1,l_2)restricted Wythoff	560	408(0)	**532**(124)	29.58(29.58)	**11.05**(**3.67**)	**6.80**(**6.80**)	76.01(34.99)
Total	2924	1933(1)	**2691**(759)	37.54(37.56)	**27.52**(**3.21**)	**6.73**(**6.73**)	72.72(38.87)

Table 20: Results of synthesizing winning formulas for Wythoff games under misère rule

Category	Test cases	Strategy					
		Solved		Time		Size	
		SynEnum	SynMS	SynEnum	SynMS	SynEnum	SynMS
Wythoff	21	10(1)	**16**(7)	**1.56**(**1.42**)	21.65(6.15)	**31.40**(**28.22**)	278.56(127.89)
l-Wythoff	140	80(8)	**102**(30)	**5.62**(**3.12**)	663.64(580.49)	**54.11**(**48.61**)	790.93(343.90)
Monotonic l-Diet Wythoff	105	61(7)	**95**(41)	**0.44**(0.49)	0.50(**0.16**)	**22.03**(**24.89**)	419.32(76.31)
Wythoff with subtraction set S	315	200(5)	**266**(71)	**1.70**(**1.62**)	25.91(7.60)	**30.09**(**29.21**)	368.59(166.89)
Wythoff with a function f	168	116(1)	**140**(25)	**3.54**(**3.55**)	442.66(384.41)	**52.01**(**51.94**)	598.21(483.18)
1st(l_1,l_2)restricted Wythoff	315	214(5)	**264**(55)	**2.38**(**2.29**)	16.59(10.67)	**37.33**(**36.46**)	365.79(238.08)
2nd(l_1,l_2)restricted Wythoff	600	440(41)	**538**(139)	**2.52**(**2.68**)	705.10(499.45)	**43.83**(**46.91**)	328.33(242.01)
3rd(l_1,l_2)restricted Wythoff	700	404(40)	**499**(135)	**12.26**(**7.91**)	723.65(662.59)	**59.64**(**55.02**)	562.53(358.20)
4th(l_1,l_2)restricted Wythoff	560	394(7)	**500**(113)	**71.94**(**2.68**)	105.31(51.97)	**31.03**(**31.03**)	474.15(316.60)
Total	2924	1919(115)	**2420**(616)	**18.84**(**3.58**)	386.13(305.07)	**42.58**(**41.65**)	453.61(286.79)

Table 21: Results of synthesizing winning strategies for Wythoff games under misère rule

Comparison on Chomp game. Table 22 presents the results for synthesizing winning formulas in the Chomp game under the misère rule. Out of 8 test cases, SynEnum solves 2 formula cases, while SynMS solves 3, with success rates of 25.0% and 37.5%, respectively. Thus SynMS solves 1 more formula cases than SynEnum. The average formula synthesis time is 17.48 seconds for SynEnum and 4.66 seconds for SynMS. Table 23 summarizes the corresponding results for synthesizing winning strategies. SynEnum solves 2 strategy cases, whereas SynMS solves 2 strategy cases. Both approaches solve 2 strategy cases



(a) Cumulative solved formula cases

(b) Cumulative solved strategy cases

Figure 10: Cumulative solved test cases over time for Wythoff game under misère rule

in this benchmark group. The average strategy synthesis time is 3.33 seconds for SynEnum and 5.17 seconds for SynMS. In terms of solution size, the average formula size is 6.00 for SynEnum and 62.33 for SynMS, and the average strategy size is 45.50 for SynEnum and 24.00 for SynMS. Figure 11 shows the cumulative number of solved test cases over time using the same solved-case filtering as the tables.

Category	Test cases	Formula					
		Solved		Time		Size	
		SynEnum	SynMS	SynEnum	SynMS	SynEnum	SynMS
Chomp	7	1(0)	**2**(1)	**1.49**(**1.49**)	6.09(1.75)	**3.00**(**3.00**)	79.50(40.00)
L-shaped Chomp	1	**1**(0)	**1**(0)	33.46(33.46)	**1.81**(**1.81**)	**9.00**(**9.00**)	28.00(28.00)
Total	8	2(0)	**3**(1)	17.48(17.48)	**4.66**(**1.78**)	**6.00**(**6.00**)	62.33(34.00)

Table 22: Results of synthesizing winning formulas for Chomp games under misère rule

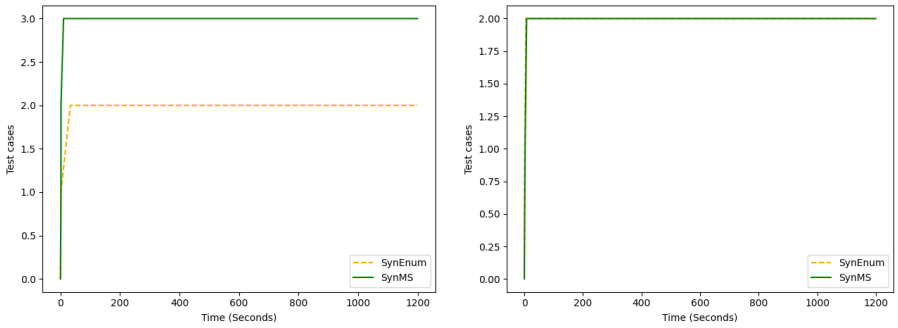
8. Related work

8.1. Analysis of ICGs

Wythoff's game has losing positions $(\lfloor n\varphi \rfloor, \lfloor n\varphi^2 \rfloor)$ for $n \geq 0$, with $\varphi = (1+\sqrt{5})/2$ [6]. Two-row Chomp has the empty board and $[m, m-1]$ for $m \geq 1$

Category	Test cases	Strategy					
		Solved		Time		Size	
		SynEnum	SynMS	SynEnum	SynMS	SynEnum	SynMS
Chomp	7	**1**(0)	**1**(0)	**5.28**(**5.28**)	7.79(7.79)	65.00(65.00)	**14.00**(**14.00**)
L-shaped Chomp	1	**1**(0)	**1**(0)	**1.38**(**1.38**)	2.54(2.54)	**26.00**(**26.00**)	34.00(34.00)
Total	8	**2**(0)	**2**(0)	**3.33**(**3.33**)	5.17(5.17)	45.50(45.50)	**24.00**(**24.00**)

Table 23: Results of synthesizing winning strategies for Chomp games under misère rule



(a) Cumulative solved formula cases

(b) Cumulative solved strategy cases

Figure 11: Cumulative solved test cases over time for Chomp game under misère rule

as its losing positions, with the two entries giving the row lengths [55]. Coloring games are impartial graph games in which two players alternately color previously uncolored vertices with one of k colors. Each move colors exactly one vertex. A player with no legal move loses under the normal-play convention [12]. A weak coloring game requires every non-isolated vertex to have a neighbor of a different color. In Weak 2-coloring game, two adjacent vertices may share a color provided that each also has a neighbor of the other color; under this rule, an initially uncolored path or cycle H is a losing starting state whenever $|V(H)| \equiv 0 \pmod 2$, where $|V(H)|$ is the number of vertices [12].

End-Nim permits reductions only to the first or last pile in a row; among equal-sized positions, those with an even number of piles are losing [44]. Using $\oplus$ for bitwise XOR, when the preceding player chooses an even-removal constraint for the next move, Odd-or-Even Nim is losing when $x_1 \oplus \cdots \oplus x_n \in \{0, 1\}$. When

an odd-removal constraint is chosen, all piles must be even and their nim-sum must be zero [42]. Greedy Nim restricts play to a largest pile; its losing states are those in which the largest size occurs an even number of times [41]. Losing positions in modulo-d one-blocking Nim satisfy $\bigoplus_i \lfloor x_i/2 \rfloor = 0$ and either forbid removals congruent to 1 modulo d or have only even pile sizes [43]. Circular Nim $CN(n,k)$ arranges n piles in a circle and allows a move to reduce one or more of k consecutive piles. $CN(n,1)$ has losing states with zero nim-sum, $CN(n,n)$ has only $(0,\ldots,0)$, and $CN(n,n-1)$ has $(a,\ldots,a)$ for $a \geq 0$[46]. For $n \leq 6$, the losing states of $CN(4,2)$ are (a,b,a,b) for $a,b \geq 0$, in which opposite piles are equal. Those of $CN(5,2)$ are (a,b,c,d,b) with $a+b=c+d$, where a is the largest pile size. Those of $CN(5,3)$ are $(0,b,c,d,b)$ with $b = c+d$[46]. The losing states of $CN(6,3)$ are (a,b,c,d,e,f) with $a+b=d+e$ and $b+c=e+f$. Those of $CN(6,4)$ satisfy the same two equalities together with $a \oplus c \oplus e = 0$, where a is the smallest pile size. The losing states of $CN(6,2)$ have not yet been fully characterized [46]. Deleting emptied piles yields Shrinking Circular Nim $SCN(4,2)$, whose losing positions are $\emptyset$, (a,a,a), and (a,b,a,b) for unequal positive a,b [56]. Slow A-Nim with $A=\{1,k\}$ and odd k removes one token from either one or k piles per move. Even total token count, $\sum_i x_i \equiv 0 \pmod 2$, characterizes its losing positions [57]. Fraenkel [32] introduced an LIA-definable ICG, namely Mark game, and showed that its set of winning states is not ultimately periodic. It follows Theorem 1 that the set of winning states is not LIA-definable. Mark game is a variant of Take-away game and shares the same constraint $\mathcal{C}$ and ending condition $\mathcal{E}$. In this game, the player can take either 1 or $\lfloor \frac{v}{2} \rfloor$ chips from the table, where v denotes the number of remaining chips. The precondition and effect of action $take(k)$ in Mark game are as follows.

- $\texttt{pre}(take(k)) : v \geq 0 \wedge k \geq 1 \wedge (k=1 \vee v = 2k \vee v = 2k+1)$;
- $\texttt{eff}(take(k)) : \{\top, v, v-k\}$.

Subtraction game [37] is a generalization of the Take-away game. Berlekamp et al. [30] proved that any Subtraction game admits an LIA-definable set of winning states. Let S be a finite set of positive integers. In this game, each

player can take k chips from the pile where $k \in S$. The precondition and effect of action $take(k)$ in Subtraction game are as follows.

- $\texttt{pre}(take(k)) : (\bigvee_{s \in S} k = s) \wedge v \geq \min(S)$ where $\min(S)$ denotes the smallest integer of S;
- $\texttt{eff}(take(k)) : \{\langle \top, v, v - k \rangle\}$;

Larsson and Wästlund [58] proved that the problem of deciding whether two LIA-definable ICGs have the same set of winning states is undecidable. Larsson [59] showed that the problem of deciding whether the set of winning states for an LIA-definable ICG is 2-dimensional periodic is undecidable.

8.2. Automatic methods to two player games

Two classical methods for solving the winning strategy of finite-state ICGs are backward induction [60] and alpha-beta pruning [61, 62]. Backward induction reasons backward from the ending state to the given initial state. For games with large state spaces, the computational complexity of backward induction increases rapidly, leading to low algorithmic efficiency and making it difficult to handle complex games. Alpha-beta pruning is a well-known pruning method for minimax trees. It reduces the number of nodes in the search tree that captures the plays from the given initial state by two players. However, the effectiveness of the alpha-beta pruning method depends on the traversal order of the nodes. If the traversal order is suboptimal, the effectiveness of pruning will be impacted.

Luo and Liu [63] extended the situation calculus for formalizing ICGs and proposed a verification approach for finite-state automata strategies. The key idea is to establish an invariant for the given automaton, which plays a role similar to that of a symbolic characterization of the winning region. However, their work focuses on verification rather than automatic strategy synthesis.

Wu et al. [21] took a further step toward automatic synthesis for infinite-state ICGs by using SMT-based reasoning to construct generalized winning formulas and strategies. Their strategy synthesis procedure first partitions the winning region into arithmetic blocks by simple division rules and then attempts to assign

an action to each block. This design is elegant, but it may fail when the chosen partition is too coarse to guarantee that every block admits a proper action.

Later, Luo and Liu [64] developed a CEGIS approach for synthesizing invariant strategies. An invariant strategy is of the form $\phi?; \pi a.a; \varphi?$, meaning whenever ϕ holds, the player can execute an action to enforce φ, and whenever φ holds, executing any action makes φ true regardless of what the action chooses. In fact, the formulas ϕ and φ are the winning and the losing formulas, respectively. Strictly speaking, Luo and Liu [64]'s approach only constructs winning and losing formulas but not winning strategies that specifies which action to do in different partitions of winning states (cf. Definition 4). In addition, as reported in [64], their approach is much less efficient than Enum. Hence, we do not compare with their approach in experimental evaluation.

8.3. Sprague-Grundy theory

Sprague-Grundy theory is another foundational line of research on impartial games, distinct from explicit strategy synthesis. This theory originates from Bouton's complete mathematical analysis of Nim [5] and was generalized to short impartial games by Sprague [65] and Grundy [66]. It assigns each position s a non-negative integer $G(s)$, called its Sprague-Grundy value or nimber, defined as the minimum excluded value of the Sprague-Grundy values of all successor positions: $G(s) = \text{mex}(\{G(t) \mid t \in N(s)\})$. The Sprague-Grundy value determines whether a position is winning or losing, since a position is losing iff its value is 0. Moreover, for a disjunctive sum of independent subgames $s_1, \ldots, s_k$, the total Sprague-Grundy value is the nim-sum of the component values, namely $G(s_1) \oplus \cdots \oplus G(s_k)$.

Sprague-Grundy theory has supported both analytical and computational studies. On the analytical side, many works study the exact Sprague-Grundy function or its regularities for specific games, such as Euclid [67, 68], Wythoff's game [69], and Grundy's game [70]. Ordinary Nim assigns $G(x_1, \ldots, x_n) = x_1 \oplus \cdots \oplus x_n$ to piles of sizes $x_1, \ldots, x_n$, where $\oplus$ denotes bitwise XOR [5]. A one-pile take-away game allowing between 1 and m chips per move has

$G(v) = v \bmod (m+1)$ for pile size v [11]. Within the same family, Boros et al. [71] obtained a closed SG formula for proper Nim with at least three piles; whenever at least one pile is empty, it reduces to $G(x) = \sum_i x_i$. The Jenkyns–Mayberry classes of symmetric and matroid hypergraph Nim likewise satisfy $G_{\mathcal{H}}(x) = h_{\mathcal{H}}(x)$ when $\min_i x_i = 0$, where $\mathcal{H}$ specifies which subsets of piles may be reduced together and $h_{\mathcal{H}}(x)$ is the maximum possible number of consecutive moves [72, 73]. Exact k-Nim has n piles and requires each move to reduce exactly k of them. $G(x) = T_{n,k}(x)$ when $n < 2k$, where $T_{n,k}(x)$ is the maximum possible number of consecutive moves, and at $k = n$ this becomes $G(x) = \min_i x_i$ [74]. Euclid is played on a pair (a, b) of positive integers; each move subtracts a positive integer multiple of one entry from the other without making it negative, and reducing either entry to zero wins. Cairns, Ho, and Lengyel established an SG formula for every position; when $0 < a < b$ and $b = qa$, the integer ratio $q = b/a$ is the SG value [67]. Outside heap games, proper 2-coloring on an initially uncolored path P_n has $G(P_n) = n \bmod 2$ [12]. These results determine the SG value within the stated games or parameter regimes, although they do not cover arbitrary hypergraph Nim or Exact k-Nim with $2k < n$. Another recurring topic is the periodicity of nim-values, for example in peg duotaire [75]. These results reveal deep mathematical structure, but they are usually game specific and rely on substantial manual analysis.

On the computational side, several works aim to accelerate the recursive evaluation of Sprague-Grundy values. Lemoine and Viennot [76] computes Sprague-Grundy values by reducing candidate values to losing position tests in suitable sum games. Beling and Rogalski [61] adapt α-β style pruning to the mex computation itself, thereby reducing the search effort required for impartial games. An additional direction combines Sprague-Grundy values with search in decomposable positions. Müller [77] proposed decomposition search for Go endgames. Uiterwijk [78] applied a related idea to Cram, a domino-placement impartial game. His method combines combinatorial game theory with transposition tables and endgame databases, which are precomputed repositories of exact values for small remaining board positions. Once a searched position can be decom-

posed into smaller components or matched with a stored endgame position, the solver can reuse the known values and avoid further exhaustive search.

However, Sprague-Grundy based methods are different from ours. They primarily compute nimbers, analyze periodic structure, or support a search for particular positions. In this respect, their objectives is related to ours, since both lines of work seek information that can guide reasoning about winning and losing states. The difference is that Sprague-Grundy based methods usually compute values for given games or positions, whereas our goal is to synthesize LIA-definable winning formulas and generalized explicit winning strategies that apply to all legal states of an infinite-state ICG.

8.4. LTL synthesis

Pnueli and Rosner [79] first proposed a well-known class of games, namely *linear temporal logic (LTL) games*, where the game is represented by an automaton, and the goal is expressed by a propositional LTL formula. The objective of LTL synthesis is to synthesize a strategy that ensures goal achievement. The method proposed in [79] is an approach based on automata and has a doubly exponential time complexity. To lower the complexity, Asarin et al. [80] restrict the goal to certain fragments of LTL formulas so as to obtain synthesis approaches with polynomial time complexity, *e.g.*, safety and reachability games whose goals are of the form $\Box\phi$ and $\Diamond\phi$, respectively, where ϕ is propositional. LTL synthesis has since been extensively studied, such as in [81, 82, 83, 84, 85, 86]. The above methods only focus on games over finite states.

In general, it is undecidable for the synthesis problem of infinite-state games [87]. Several decidable classes of infinite-state games have nevertheless been identified, *e.g.*, games on automatic graphs [88], pushdown graphs [89, 90], and prefix recognizable graphs [91].

Besides automata, arithmetic formulas provide an alternative formalization of games. Compared with automata, arithmetic formulas are often easier to inspect and are therefore more suitable for giving explicit symbolic descriptions. Recently, arithmetic formulas have been used to represent infinite-state games

[92, 93, 94, 95]. Our framework is also based on arithmetic formulas and is therefore close in spirit to these works.

Beyene et al. [92] devise sound and relatively complete proof rules for computing winning strategies for safety, reachability and LTL games over infinite states. All impartial combinatorial games considered in this paper are reachability games over infinite states. The rules for reachability games rely on winning formulas and well founded relations that are used to guarantee the correctness and termination of the winning strategy respectively. The generation of winning formulas and well founded relations is solved by the EHSF solver [96]. Our approach finds the winning formula via enumeration, and does not generate a well founded relation because an impartial combinatorial game always ends after finitely many moves. One major drawback of Beyene et al.'s approach is that it requires users to provide hints in the form of templates to synthesize winning strategies, and hence it is not a full mechanization. By contrast, our approach succeeds in computing winning strategies for all games considered in this paper in a fully automated way.

Farzan and Kincaid [93] propose a fully automated method, called SimSynth, for synthesizing winning strategies for reachability games over infinite states. SimSynth first unfolds a game into a bounded variant. It then reduces the problem of synthesizing a winning strategy for the bounded variant to linear arithmetic satisfiability [97]. Finally, it attempts to generalize the strategy so as to hold for all states. The insight of our approach is different from SimSynth. Instead, we first construct the winning formula, and then generate the winning strategy from the cover of the winning formula. SimSynth does not involve the winning formula, which precisely describes the set of winning states. In practice, SimSynth is only able to synthesize the winning strategy of an ICG instance, and it does not scale up well. For example, it fails to solve the 2-piled Nim game for the state $v_1 = 6 \wedge v_2 = 6$ within 10 minutes. By contrast, our approach succeeds in computing a generalized winning strategy for the 2-piled Nim game in 3.91 seconds, and the resulting strategy applies not only to one state but to all winning states.

Baier et al. [94] study two-player reachability games represented by logical constraints. Their method uses Craig interpolation to identify necessary subgoals. Each such subgoal is a transition condition that every winning play of the reachability player must cross before reaching the final target. The game is then decomposed recursively along these subgoals. This method is close to our work in that it also reasons about winning behavior in a symbolic infinite-state setting. The main difference is that Baier et al. [94] organize solving around causal subgoals. Their subgoals are transition predicates that every play reaching the target must pass through, and the solver recursively slices the game into smaller parts before and after such predicates. Our approach does not search for intermediate causal milestones or solve a sequence of sliced subgames. Instead, it first synthesizes a single exact arithmetic formula that characterizes the whole winning region of an ICG, and then derives action rules that cover this region and move every covered winning state to a losing state.

Faella and Parlato [95] consider reachability games with a bounded safety player. Their method reduces winner determination to solving constrained Horn clauses. This work also uses arithmetic constraints to analyze infinite-state games. However, its objective is different from ours. It does not produce a generalized strategy that applies to all legal winning states of an ICG. Moreover, their framework is theory-based and not tailored to the arithmetic structure that arises in ICGs, whereas our method is designed for LIA and often requires modulo constraints.

Daniel and Oliver [98] propose a counterexample-guided framework for synthesizing safety controllers over potentially infinite graphs. Their method combines a symbolic teacher with a learner that refines a candidate winning set from positive, negative, existential, and universal counterexamples. In their implementation, games are encoded by quantifier-free first order formulas in linear integer arithmetic, and the learned winning sets are represented by decision trees. This work is particularly close in core idea to ours because both methods follow a counterexample-guided synthesis paradigm. The difference is that Daniel and Oliver [98] focus on safety objectives and learn a safe winning region.

Our method instead targets impartial combinatorial games and synthesizes an exact arithmetic winning formula. We then construct an explicit generalized winning strategy from the cover of that formula.

Neider and Topcu [88] also adopt a counterexample-guided teacher-learner framework, but within a framework based on automata theory. They formalize safety games over infinite graphs as rational safety games. Sets of states are represented by regular languages, and transition relations are represented by rational relations. Their method learns a deterministic finite automaton that accepts a winning set. Like our approach, this method seeks symbolic descriptions of winning states in infinite-state games. However, the underlying representation is different. Their output is an automaton. Our output is an interpretable arithmetic winning formula together with explicit action rules. This arithmetic representation is more natural for ICGs, whose states are usually described by integer variables and modular structure.

Samuel et al. [99] develop symbolic fixpoint algorithms for logical LTL games. Their framework lifts classical fixpoint reasoning to symbolic infinite-state settings and handles a broad range of temporal objectives. Compared with our work, the main advantage of their method is its generality. It is designed for logical LTL games rather than for ICGs in particular. Its exactness depends on the successful termination of the symbolic fixpoint computations. This condition is difficult to satisfy in infinite-state arenas. Our approach is more specialized to the structure of impartial combinatorial games. We synthesize exact arithmetic winning formulas and explicit generalized strategies composed by rules for that setting.

Heim and Dimitrova [100] tackle the non-termination problem of symbolic game solving by introducing acceleration into symbolic attractor computation. Their framework of reactive program games supports objectives such as reachability, safety, Büchi, and co-Büchi. The method summarizes unbounded strategic loops and thereby improves symbolic solving over infinite domains. This is relevant to our work because it also aims to make infinite-state game solving effective in practice. The difference is that Heim and Dimitrova [100] improve a

general fixpoint based solving framework. Their method still relies on the reactive program game formalization and on finding useful accelerations for symbolic attractor computation. Our method focus on automatically synthesizing winning formula and winning strategy of an ICG.

Schmuck et al. [101] further improve the scalability by combining symbolic infinite-state solving with finite abstractions. Their method computes permissive strategy templates on abstract games, identifies useful local subgames, and reuses localized attractor information while solving the full game. This is a sophisticated way to reduce the cost of symbolic game solving. Our method does not rely on abstraction or local subgame identification. Instead, it directly searches for exact arithmetic conditions that characterize winning and losing states in ICGs. This difference is important because the behavior of an ICG is often governed by global arithmetic regularities rather than by local subgame decompositions.

Azzopardi et al. [102] study full LTL synthesis over infinite-state arenas by means of abstraction refinement. Their framework handles very general temporal objectives and therefore has a scope well beyond that of the present paper. This level of generality is achieved by substantially heavier synthesis machinery based on predicate discovery, abstraction refinement, and repeated checks of abstract counterstrategies. The method is therefore sensitive to the quality of the abstraction and the refinement process. Our work solves a different synthesis problem. Rather than solving full LTL synthesis over general infinite-state arenas, we exploit the arithmetic structure of ICGs to synthesize exact winning formulas and explicit generalized winning strategies.

8.5. Generalized planning

A related problem is generalized planning that aims to construct a generalized planning that works for possibly infinitely many planning instances sharing the similar structure [103]. Levesque [103] implemented an iterative planner called KPLANNER. The core idea is to first search for planning solutions with no loop structure and variable values less than a threshold, and then identify the

common structure in the planning solution based on the built-in pattern matching mechanism of the Prolog language, and extract the planning program with loop structure. On this basis, Hu and Levesque [104] proposed a new planner FSAPLANNER. Compared with KPLANNER, FSAPLANNER proposed a more expressive planning solution form: finite-state automata (FSA). At the same time, they proposed that the existence of planning solutions for one-dimensional planning problems is decidable. Later, Hu and Giacomo [105] analyzed the complexity of the problem and proved that it is complete in exponential space. However, the above methods are mainly aimed at single variable scenarios and are difficult to directly extend to complex planning problems with multiple variables.

To solve the multivariate general planning problem, Srivastava et al. [106] proposed a method to generate a FSA plan based on state abstraction using 3-valued logic. The method can be applied to loop structures, but the generated solution is not guaranteed to achieve the goal. To solve this problem, they designed an algorithm to calculate the conditions for the planning to terminate and achieve the goal when it only contains simple loop structures, namely applicability condition[107]. Nevertheless, the planners synthesized by their method are only allowed to have simple loop structures and cannot synthesize planners with nested loops.

Compiling a generalized planning problem into another planning problem and then indirectly solving the original one is also an effective method. Bonet et al. [108] proposed a method to automatically construct a finite state controller (FSC). This method transforms a partially observable planning problem into a consistent planning problem, and then uses a classical planning solver to indirectly solve the consistent planning problem. However, it is not clear which variants of the original problem the generated FSC can solve. Bonet and Geffner [109] clarify which original problems the FSC generated by Bonet et al. [108] can solve by characterizing the general structure that supports policy generalization. Segovia et al. codify a finite number of generalized planning problems with the same problem domain into a single classical planning problem [110, 111, 112]. However, their approach is only applicable to classical planning problems. Sri-

vastava et al. [113] proposed a new type of planning problem: Qualitative Numeric Planning (QNP). QNP imposes two restrictions on Numerical Planning: (1) Any formula is a Boolean combination of simple numeric literals in the form of $v > 0$ or $v = 0$; (2) The value of the variable decreases or increases by an uncertain amount. Under these two constraints, the state space of the QNP can be compressed to a finite space of size $2^{|V|}$, where $|V|$ is the number of numerical variables. Therefore, the QNP problem is decidable. But the expressive power of the QNP proposed above is relatively limited. Bonet et al. [114] used SAT solvers and description logic to learn an abstract QNP from a finite number of problem instances. However, the learned QNP may not be a valid abstraction for the entire set of original problem instances. Later, Bonet et al. [115] studied the necessary conditions to ensure that an abstraction is valid, that is, which problem instances can guarantee that the abstraction is valid.

The abstraction-refinement framework is an effective approach for verifying the properties of multi agent systems. The framework first builds an abstract model based on 3-value semantics to preliminarily judge the system properties. If the property is true or false in the abstract model, the truth or falsehood of the property in the specific model can be directly determined; if it cannot be determined, it means that the abstract model is too rough and cannot accurately reflect the satisfaction of the property. At this point, the framework refines the abstract model by searching for failed states until a clear conclusion can be drawn. In order to achieve this process, a variety of abstraction and refinement strategies have been proposed based on this framework [116, 117, 118, 119]. Nevertheless, the abstraction-refinement framework cannot guarantee a complete abstract model, because it is undecidable to find the failure state in these two logics. In addition, the above works all focus on the abstraction of the model rather than the theoretical abstraction.

In this paper, we concentrate on ICGs that is more complex than the planning domain. The former involves two players while the latter only focuses on the single agent case. Hence it is necessary to consider not only how the current player chooses actions, but also how the other player responds when we

synthesize winning strategies. Recently, De Giacomo et al. [120] study the translation from generalized planning under partial observability to two-player games with perfect information. In the case there are no additional fairness assumptions, the former corresponds to two-player games with imperfect information. By adapting the belief-state construction, De Giacomo et al. remove imperfect information from games, and hence obtain a two-player game with perfect information corresponding to the given generalized planning problem. However, they do not provide an approach to synthesizing winning strategies for the games with perfect information corresponding to the given generalized planning problem.

9. Conclusions and future work

We have devised an effective approach to computing generalized winning strategies that works for possibly infinite many instances of ICGs. The key insight of our approach is to generate a candidate formula that is consistent with two generated sets of positive states and negative states. To this end, we have put forward two approach to solve this problem. One adapts an enumerative method, and the other improves efficiency by reducing the synthesis problem as an exact binary classification. Then, by integrating the SMT-based verification procedure, we have developed a CEGIS approach to synthesizing the winning formula and strategy of ICGs. We have implemented our approach and experimental results show the effectiveness and scalability of our proposed approach.

Our approach have some limitations, and hence leading to several avenues for future work. Firstly, the ICGs considered in this paper are formalized in LIA. For other ICGs, their formalization involves real numbers, existential quantifiers, multiple operator, rounding operator, exclusive disjunction operator and so on. We would also like to consider the games whose formalizations require more expressive language. Secondly, for the same solvable test cases, the strategy generated based on exact binary classification is larger than the one generated by the enumeration method, while the winning strategy can be represented in a more concise form. We would like to design the solution to strategy synthesis

based on more efficient algorithms. Finally, we focus on ICGs. It is also interesting to apply our approach to other games, *e.g.*, partizan combinatorial games (*i.e.*, some moves are available to one player and not to the other).

References

[1] K. Chatterjee, T. A. Henzinger, N. Piterman, Strategy logic, Information and Computation 208 (2010) 677–693.

[2] D. Zhang, M. Thielscher, Representing and Reasoning about Game Strategies, Journal of Philosophical Logic 44 (2015) 203–236.

[3] L. Xiong, Y. Liu, Strategy Representation and Reasoning for Incomplete Information Concurrent Games in the Situation Calculus, in: Proceedings of the Twenty-Fifth International Joint Conference on Artificial Intelligence (IJCAI-2016), 2016, pp. 1322–1329.

[4] T. S. Ferguson, Game Theory, World Scientific, 2018.

[5] C. L. Bouton, Nim, a Game with a Complete Mathematical Theory, Annals of Mathematics 3 (1901) 35–39.

[6] W. A. Wythoff, A modification of the game of nim, Nieuw Archief voor Wiskunde 7 (1907) 199–202.

[7] F. Schuh, The Game of Divisions, Nieuw Tijdschrift voor Wiskunde 39 (1952) 299–304.

[8] T. S. Ferguson, Some Chip Transfer Games, Theoretical Computer Science 191 (1998) 157–171.

[9] W. Liu, M. Zhao, General Restrictions of Wythoff-like Games, Theoretical Computer Science 602 (2015) 80–88.

[10] J. Ahn, B. Chen, R. Chen, E. Erives, J. Fleming, M. Gerovitch, T. Gopalakrishna, T. Khovanova, N. Malur, N. Polina, P. Sahoo, On

Variations of Nim and Chomp, Minnesota Journal of Undergraduate Mathematics 4 (2018) 1–11.

[11] S. W. Golomb, A mathematical investigation of games of “take-away", Journal of Combinatorial Theory 1 (1966) 443–458.

[12] G. Beaulieu, K. Burke, E. Duchêne, Impartial Coloring Games, Theoretical Computer Science 485 (2013) 49–60.

[13] E. Sopena, *i*-Mark: A new subtraction division game, Theoretical Computer Science 627 (2016) 90–101.

[14] V. Gurvich, N. B. Ho, On tame, pet, domestic, and miserable impartial games, Discrete Applied Mathematics 243 (2018) 54–72.

[15] D. de Jonge, D. Zhang, Lifted Backward Search for General Game Playing, in: Proceedings of the Twenty-Ninth Australasian Joint Conference on Advances in Artificial Intelligence (AI-2016), 2016, pp. 3–16.

[16] M. Fox, D. Long, PDDL2.1 : An Extension to PDDL for Expressing Temporal Planning Domains, Journal of Artificial Intelligence Research 20 (2003) 61–124.

[17] A. Udupa, A. Raghavan, J. V. Deshmukh, S. Mador-Haim, M. M. K. Martin, R. Alur, TRANSIT: Specifying Protocols with Concolic Snippets, in: Proceedings of the Thirty-Fourth Annual ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI-2013), 2013, pp. 287–296.

[18] L. D. Moura, N. Bjørner, Z3: an Efficient SMT Solver, in: Proceedings of the Fourteenth International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS-2008), volume 4963 of *Lecture Notes in Computer Science*, Springer, 2008, pp. 337–340.

[19] C. Barrett, C. L. Conway, M. Deters, L. Hadarean, D. Jovanovic, T. King, A. Reynolds, C. Tinelli, CVC4, in: Proceedings of the Twenety-Third

International Conference on Computer Aided Verification (CAV-2011), volume 6806 of *Lecture Notes in Computer Science*, Springer, 2011, pp. 171–177.

[20] A. Cimatti, A. Griggio, B. J. Schaafsma, R. Sebastiani, The MathSAT5 SMT Solver, in: Proceedings of the Nineteenth International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS-2013), volume 7795 of *Lecture Notes in Computer Science*, Springer, 2013, pp. 93–107.

[21] K. Wu, L. Fang, L. Xiong, Z. Lai, Y. Qiao, K. Chen, F. Rong, Automatic Synthesis of Generalized Winning Strategies of Impartial Combinatorial Games Using SMT Solvers, in: Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence (IJCAI-2020), 2020, pp. 1703–1711.

[22] L. Fang, M. Yang, D. Cheng, Y. Hao, Q. Guan, L. Xiong, Generalized Strategy Synthesis of Infinite-State Impartial Combinatorial Games via Exact Binary Classification, in: Proceedings of the Twenty-Third International Conference on Autonomous Agents and Multiagent Systems (AAMAS-2024), 2024, pp. 562–570.

[23] D. C. Cooper, Theorem Proving in Arithmetic without Multiplication, Machine Intelligence 7 (1972) 300.

[24] D. Monniaux, Quantifier Elimination by Lazy Model Enumeration, in: Proceedings of the Twenty-Second International Conference on Computer Aided Verification (CAV-2010), volume 6174 of *Lecture Notes in Computer Science*, Springer, 2010, pp. 585–599.

[25] M. Presburger, Über die vollständigkeit eines gewissen systems der arithmetik ganzer zahlen, in welchem die addition als einzige operation hervortritt, in: Comptes Rendus du I congrès de Mathématiciens des Pays Slaves, 1929, pp. 92–101.

[26] S. Ginsburg, E. Spanier, Semigroups, Presburger formulas, and languages, Pacific Journal of Mathematics 16 (1966) 285–296.

[27] S. Feng, L. Fang, J. Luo, Q. Guan, Decidability and Undecidability Results for LIA-Definable Impartial Combinatorial Games, arXiv preprint arXiv:2606.25276 (2026). URL: https://arxiv.org/abs/2606.25276. doi:10.48550/arXiv.2606.25276.

[28] E. Zermelo, Über eine anwendung der mengenlehre auf die theorie des schachspiels, in: Proceedings of the Fifth International Congress of Mathematicians Conference (ICM-1913), 1913, pp. 501–504.

[29] D. Gale, F. M. Stewart, Infinite games with perfect information, in: H. W. Kuhn, A. W. Tucker (Eds.), Contributions to the Theory of Games, volume 2 of *Annals of Mathematics Studies*, Princeton University Press, 1953, pp. 245–266.

[30] E. R. Berlekamp, J. H. Conway, R. K. Guy, Winning Ways for Your Mathematical Plays, 2nd ed., AK Peters/CRC Press, 2001.

[31] M. I. Krusemeyer, 2009 canada/usa mathcamp qualifying quiz, problem 6, Canada/USA Mathcamp, 2009. URL: https://mathcamp.org/2009/quiz/.

[32] A. S. Fraenkel, Aperiodic subtraction games, Electronic Journal of Combinatorics 18 (2011) 1–12.

[33] J. Hoffmann, R. I. Brafman, Conformant planning via heuristic forward search: A new approach, Artificial Intelligence 170 (2006) 507–541.

[34] J. Rintanen, Planning as satisfiability: Heuristics, Artificial Intelligence 193 (2012) 45–86.

[35] A. Lévy, Basic Set Theory, Perspectives in Mathematical Logic, Springer-Verlag, Berlin, 1979.

[36] Y. Chu, S. Cai, C. Luo, NuWLS: Improving Local Search for (Weighted) Partial MaxSAT by New Weighting Techniques, in: Proceedings of the

Thirty-Seventh AAAI Conference on Artificial Intelligence (AAAI-2023), 2023, pp. 3915–3923.

[37] I. M. Yaglom, Two games with matchsticks, Kvant Selecta: Combinatorics, I 1 (2001) 1–8.

[38] C. P. Welter, The advancing operation in a special abelian group, Indagationes Mathematicae 55 (1952) 304–314.

[39] G. Cairns, N. B. Ho, Min, A Combinatorial Game Having a Connection With Prime Numbers, Integers 10 (2010) 765–770.

[40] N. B. Ho, Three-pile Sharing Nim and the Quadratic Time Winning Strategy, Theoretical Computer Science 646 (2016) 49–60.

[41] M. Albert, R. J. Nowakowski, Nim Restrictions, Integers: Electronic Journal of Combinatorial Number Theory 4 (2004) 1–10.

[42] F. Smith, P. Stănică, Comply/Constrain Games or Games with a Muller Twist, Integers: Electronic Journal of Combinatorial Number Theory 2 (2002) 1–11.

[43] H. Gavel, P. Strimling, Nim with a Modular Muller Twist, Integers: Electronic Journal of Combinatorial Number Theory 4 (2004) 1–9.

[44] M. H. Albert, R. J. Nowakowski, The Game of End-Nim, Electronic Journal of Combinatorics 8 (2001) 1–12.

[45] R. Sprague, Über zwei abarten von nim, Tohoku Mathematical Journal 43 (1937) 351–354.

[46] M. Dufour, S. Heubach, Circular Nim Games, The Electronic Journal of Combinatorics 20 (2013) 1–26.

[47] J. Kahane, A. S. Fraenkel, k-Welter–A Generalization of Welter's Game, Journal of Combinatorial Theory, Series A 46 (1987) 1–20.

[48] N. B. Ho, The Max–Welter Game, Discrete Mathematics 318 (2014) 41–47.

[49] A. S. Fraenkel, How to beat your wythoff games' opponent on three fronts, The American Mathematical Monthly 89 (1982) 353–361.

[50] E. Duchene, A. Fraenkel, S. Gravier, R. J. Nowakowski, Another bridge between Nim and Wythoff, The Australasian Journal of Combinatorics 44 (2009) 43–56.

[51] A. S. Fraenkel, N. B. Ho, When are Translations of P-positions of Wythoff's Game P-positions?, Integers 15A (2015) 1–12.

[52] V. Gurvich, Further Generalizations of the Wythoff Game and the Minimum Excludant, Discrete Applied Mathematics 160 (2012) 941–947.

[53] W. Liu, H. Li, General Restriction of (s, t)-Wythoff's Game, The Electronic Journal of Combinatorics 21 (2014) 2–44.

[54] D. Eric, G. Sylvain, Geometrical Extensions of Wythoff's game, Discrete Mathematics 309 (2009) 3595–3608.

[55] A. E. Brouwer, G. Horváth, I. Molnár-Sáska, C. Szabó, On Three-Rowed Chomp, Integers: Electronic Journal of Combinatorial Number Theory 5 (2005) 1–11.

[56] H. Oginuma, M. Shinoda, Shrinking Circular Nim, Journal of Information Processing 33 (2025) 1064–1070.

[57] M. Dufour, S. Heubach, On the $\mathcal{P}$-Positions of Some Infinite Families of Slow A-Nim, Integers: Electronic Journal of Combinatorial Number Theory 26 (2026) 1–25.

[58] U. Larsson, J. Wästlund, From heaps of matches to the limits of computability, The Electronic Journal of Combinatorics 20 (2013) 1–12.

[59] U. Larsson, Impartial games emulating one-dimensional cellular automata and undecidability, Journal of Combinatorial Theory, Series A 120 (2013) 1116–1130.

[60] J. Watson, Strategy: an introduction to game theory, Third ed., W. W. Norton & Company, 2002.

[61] P. Beling, M. Rogalski, On Pruning Search Trees of Impartial Games, Artificial Intelligence 283 (2020) 103262.

[62] G. T. Heineman, G. Pollice, S. Selkow, Algorithms in a Nutshell, O'Reilly Media, 2008.

[63] K. Luo, Y. Liu, Automatic Verification of FSA Strategies via Counterexample-Guided Local Search for Invariants, in: Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence (IJCAI-2019), 2019, pp. 1814–1821.

[64] K. Luo, Y. Liu, Automated Synthesis of Generalized Invariant Strategies via Counterexample-Guided Strategy Refinement, in: Proceedings of the Thirty-Sixth AAAI Conference on Artificial Intelligence (AAAI-2022), 2022, pp. 5800–5808.

[65] R. Sprague, Über mathematische kampfspiele, Tohoku Mathematical Journal, First Series 41 (1935) 438–444.

[66] P. M. Grundy, Mathematics and games, Eureka 2 (1939) 6–8.

[67] G. Cairns, N. B. Ho, T. Lengyel, The sprague-grundy function of the real game euclid, Discrete Mathematics 311 (2011) 457–462.

[68] G. Nivasch, The sprague-grundy function of the game euclid, Discrete Mathematics 306 (2006) 2798–2800.

[69] U. Blass, A. S. Fraenkel, The sprague-grundy function for wythoff's game, Theoretical computer science 75 (1990) 311–333.

[70] R. K. Guy, C. A. Smith, The g-values of various games, in: Mathematical Proceedings of the Cambridge Philosophical Society, volume 52, Cambridge University Press, 1956, pp. 514–526.

[71] E. Boros, V. Gurvich, N. B. Ho, K. Makino, On the Sprague–Grundy Function of Extensions of Proper Nim, International Journal of Game Theory 50 (2021) 635–654.

[72] E. Boros, V. Gurvich, N. B. Ho, K. Makino, P. Mursic, Sprague–Grundy Function of Symmetric Hypergraphs, Journal of Combinatorial Theory, Series A 165 (2019) 176–186.

[73] E. Boros, V. Gurvich, N. B. Ho, K. Makino, P. Mursic, Sprague–Grundy Function of Matroids and Related Hypergraphs, Theoretical Computer Science 799 (2019) 40–58.

[74] E. Boros, V. Gurvich, N. B. Ho, K. Makino, P. Mursic, On the Sprague–Grundy Function of Exact k-Nim, Discrete Applied Mathematics 239 (2018) 1–14.

[75] J. Grossman, Periodicity in one-dimensional peg duotaire, Theoretical Computer Science 313 (2004) 417–425.

[76] J. Lemoine, S. Viennot, Nimbers are inevitable, Theoretical Computer Science 462 (2012) 70–79.

[77] M. Müller, Decomposition search: A combinatorial games approach to game tree search, with applications to solving go endgames, in: IJCAI, 1999, pp. 578–583.

[78] J. W. Uiterwijk, Solving cram using combinatorial game theory, in: Advances in Computer Games, Springer, 2019, pp. 91–105.

[79] A. Pnueli, R. Rosner, On the Synthesis of a Reactive Module, in: Proceedings of the Sixteenth ACM SIGPLAN-SIGACT Symposium Conference on Principles of programming languages (POPL-1989), 1989, pp. 179–190.

[80] E. Asarin, O. Maler, A. Pnueli, J. Sifakis, Controller Synthesis for Timed Automata, in: Proceedings of the Fifth International Federation of Automatic Control Conference on System Structure and Control (SSC-1998), 1998, pp. 447–452.

[81] E. Niklas, L. Alexander, N. Nina, R. Leonid, SAT-based Strategy Extraction in Reachability Games, in: Proceedings of the Twenty-Ninth International Conference on Artificial Intelligence (AAAI-2015), 2015, pp. 3738–3745.

[82] S. Zhu, L. M. Tabajara, J. Li, G. Pu, M. Y. Vardi, Symbolic LTL_f Synthesis, in: Proceedings of the Twenty-Sixth International Joint Conference on Artificial Intelligence (IJCAI-2017), 2017, pp. 1362–1369.

[83] A. Camacho, C. Muise, J. A. Baier, S. A. McIlraith, LTL Realizability via Safety and Reachability Games, in: Proceedings of the Twenty-Seventh International Joint Conference on Artificial Intelligence (IJCAI-2018), 2018, pp. 4683–4691.

[84] G. D. Giacomo, M. Favorito, J. Li, M. Y. Vardi, S. Xiao, S. Zhu, LTL_f Synthesis as AND-OR Graph Search: Knowledge Compilation at Work, in: Proceedings of the Thirty-First International Joint Conference on Artificial Intelligence (IJCAI-2022), 2022, pp. 2591–2598.

[85] M. Balachander, E. Filiot, J. Raskin, LTL Reactive Synthesis with a Few Hints, in: Proceedings of the Twenty-Ninth International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS-2023), volume 13994 of *Lecture Notes in Computer Science*, Springer, 2023, pp. 309–328.

[86] J. Kretínský, T. Meggendorfer, M. Prokop, S. Rieder, Guessing Winning Policies in LTL Synthesis by Semantic Learning, in: Proceedings of the Thirty-Fifth International Conference on Computer Aided Verification (CAV-2023), volume 13964 of *Lecture Notes in Computer Science*, Springer, Cham, 2023, pp. 390–414.

[87] L. de Alfaro, T. A. Henzinger, R. Majumdar, Symbolic Algorithms for Infinite-State Games, in: Proceedings of the Twelfth International Conference on Computer Aided Verification (CAV-2001), volume 2102 of *Lecture Notes in Computer Science*, Springer, 2001, pp. 536–550.

[88] D. Neider, U. Topcu, An Automaton Learning Approach to Solving Safety Games over Infinite Graphs, in: Proceedings of the Twenty-second International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS-2016), volume 9636 of *Lecture Notes in Computer Science*, Springer, 2016, pp. 204–221.

[89] T. Cachat, Symbolic Strategy Synthesis for Games on Pushdown Graphs, in: Proceedings of the Twenty-ninth International Colloquium on Automata, Languages and Programming (ICALP-2002), volume 2380 of *Lecture Notes in Computer Science*, Springer, 2002, pp. 704–715.

[90] K. Chatterjee, N. Fijalkow, Infinite-state games with finitary conditions, in: Proceedings of the Thirty-First EACSL Annual Conference on Computer Science Logic (CSL-2013), Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2013, pp. 181–196.

[91] T. Cachat, Uniform Solution of Parity Games on Prefix-Recognizable Graphs, Electronic Notes in Theoretical Computer Science 68 (2003) 71–84.

[92] T. Beyene, S. Chaudhuri, C. Popeea, A. Rybalchenko, A Constraint-based Approach to Solving Games on Infinite Graphs, in: Proceedings of the Forty-First ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL-2014), 2014, pp. 221–233.

[93] A. Farzan, Z. Kincaid, Strategy Synthesis for Linear Arithmetic Games, Proceedings of the ACM on Programming Languages 2 (2017) 1–30.

[94] C. Baier, N. Coenen, B. Finkbeiner, F. Funke, S. Jantsch, J. Siber, Causality-based game solving, in: Proceedings of the Thirty-Third Inter-

national Conference on Computer Aided Verification (CAV-2021), volume 12759 of *Lecture Notes in Computer Science*, Springer, 2021, pp. 894–917.

[95] M. Faella, G. Parlato, Reachability Games Modulo Theories with a Bounded Safety Player, in: Proceedings of the Thirty-Seventh AAAI Conference on Artificial Intelligence (AAAI-2023), 2023, pp. 6330–6337.

[96] T. A. Beyene, C. Popeea, A. Rybalchenko, Solving Existentially Quantified Horn Clauses, in: Proceedings of the Twenty-Fifth International Conference on Computer Aided Verification (CAV-2013), volume 8044 of *Lecture Notes in Computer Science*, Springer, 2013, pp. 869–882.

[97] A. Farzan, Z. Kincaid, Linear Arithmetic Satisfiability via Strategy Improvement, in: Proceedings of the Twenty-Fifth International Joint Conference on Artificial Intelligence (IJCAI-2016), 2016, pp. 735–743.

[98] N. Daniel, M. Oliver, Learning-based Synthesis of Safety Controllers, in: Proceedings of the Nineteenth International Conference on Formal Methods in Computer Aided Design (FMCAD-2019), IEEE, 2019, pp. 120–128.

[99] S. Samuel, D. D'Souza, R. Komondoor, Symbolic Fixpoint Algorithms for Logical LTL Games, in: Proceedings of the Thirty-Eighth IEEE/ACM International Conference on Automated Software Engineering (ASE-2023), IEEE, 2023, pp. 698–709.

[100] P. Heim, R. Dimitrova, Solving Infinite-state Games via Acceleration, Proceedings of the ACM on Programming Languages 8 (2024) 1696–1726.

[101] A.-K. Schmuck, P. Heim, R. Dimitrova, S. P. Nayak, Localized Attractor Computations for Infinite-State Games, in: Proceedings of the Thirty-Sixth International Conference on Computer Aided Verification (CAV-2024), volume 14683 of *Lecture Notes in Computer Science*, Springer, 2024, pp. 135–158.

[102] S. Azzopardi, N. Piterman, L. Di Stefano, G. Schneider, Full ltl synthesis over infinite-state arenas, in: Proceedings of the Thirty-Seventh International Conference on Computer Aided Verification (CAV-2025), volume 15934 of *Lecture Notes in Computer Science*, Springer, 2025, pp. 274–297.

[103] H. J. Levesque, Planning with Loops, in: Proceedings of the Nineteenth International Joint Conference on Artificial Intelligence (IJCAI-2005), 2005, pp. 509–515.

[104] Y. Hu, H. J. Levesque, A Correctness Result for Reasoning about One-Dimensional Planning Problems, in: Proceedings of the Twelfth International Conference on Principles of Knowledge Representation and Reasoning (KR-2010), 2010, pp. 362–371.

[105] Y. Hu, G. D. Giacomo, Generalized Planning: Synthesizing Plans that Work for Multiple Environments, in: Proceedings of the Twenty-Second International Joint Conference on Artificial Intelligence (IJCAI-2011), 2011, pp. 918–923.

[106] S. Srivastava, N. Immerman, S. Zilberstein, A new representation and associated algorithms for generalized planning, Artificial Intelligence 175 (2011) 615–647.

[107] S. Srivastava, N. Immerman, S. Zilberstein, Applicability conditions for plans with loops: Computability results and algorithms, Artificial Intelligence 191 (2012) 1–19.

[108] B. Bonet, H. Palacios, H. Geffner, Automatic Derivation of Memoryless Policies and Finite-State Controllers Using Classical Planners, in: Proceedings of the Nineteenth International Conference on Automated Planning and Scheduling (ICAPS-2009), 2009, pp. 34–41.

[109] B. Bonet, H. Geffner, Policies that Generalize: Solving Many Planning Problems with the Same Policy, in: Proceedings of the Twenty-Fourth In-

ternational Joint Conference on Artificial Intelligence (IJCAI-2015), 2015, pp. 2798–2804.

[110] J. Segovia, S. Jiménez, A. Jonsson, Generalized planning with procedural domain control knowledge, in: Proceedings of the International Conference on Automated Planning and Scheduling, 2016, pp. 285–293.

[111] J. Segovia-Aguas, S. Jiménez, A. Jonsson, Computing programs for generalized planning using a classical planner, Artificial Intelligence 272 (2019) 52–85.

[112] J. Segovia-Aguas, S. J. Celorrio, A. Jonsson, Generalized Planning as Heuristic Search, in: Proceedings of the Thirty-First International Conference on Automated Planning and Scheduling (ICAPS-2021), 2021, pp. 569–577.

[113] S. Srivastava, S. Zilberstein, N. Immerman, H. Geffner, Qualitative Numeric Planning, in: Proceedings of the Twenty-Fifth AAAI Conference on Artificial Intelligence (AAAI-2011), 2011, pp. 1010–1016.

[114] B. Bonet, G. Francès, H. Geffner, Learning Features and Abstract Actions for Computing Generalized Plans, in: Proceedings of the Thirty-Third AAAI Conference on Artificial Intelligence (AAAI-2019), 2019, pp. 2703–2710.

[115] B. Bonet, R. Fuentetaja, Y. E-Martín, D. Borrajo, Guarantees for Sound Abstractions for Generalized Planning, in: Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence (IJCAI-2019), 2019, pp. 1566–1573.

[116] A. Lomuscio, J. Michaliszyn, An Abstraction Technique for the Verification of Multi-Agent Systems Against ATL Specifications, in: Proceedings of the Fourteenth International Conference on Principles of Knowledge Representation and Reasoning (KR-2014), 2014, pp. 428–437.

[117] A. Lomuscio, J. Michliszyn, Verification of multi-agent systems via predicate abstraction against atlk specifications, in: Proceedings of the Eighteen International Conference on Autonomous Agents and Multiagent Systems (AAMAS-2016), 2016, pp. 662–670.

[118] F. Belardinelli, A. Lomuscio, J. Michaliszyn, Agent-based refinement for predicate abstraction of multi-agent systems, in: Proceedings of the Twenty-Second European Conference on Artificial Intelligence (ECAI-2016), 2016, pp. 286–294.

[119] F. Belardinelli, A. Ferrando, V. Malvone, An abstraction-refinement framework for verifying strategic properties in multi-agent systems with imperfect information, Artificial Intelligence 316 (2023) 103847.

[120] G. De Giacomo, A. Murano, S. Rubin, A. D. Stasio, Imperfect-Information Games and Generalized Planning, in: Proceedings of the Twenty-fifth International Joint Conference on Artificial Intelligence (IJCAI-2016), 2016, pp. 1037–1043.